

第二部分·AI Agent 编程

第 9 章

AI Agent 编程基础

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

9. AI Agent 编程基础

本页是**第二部分（AI Agent 编程）**第 9 章的章首页。基线版本：dsh 0.1.5-rc.2 (npm latest, 2026-09-10 发布)；教材核对日期 2026-09-19；维护件见 docs/build/dsh_version.yaml。

本章概览

第一部分教会你用 NumPy、SciPy、Pandas、Matplotlib、NetworkX、Statsmodels、scikit-learn 做科学计算；从本章开始，我们换一个视角：**当 AI 能自己读文件、跑命令、改代码、看报错时，科学计算的工作方式会变成什么样？**

本章以 DeepSeek Harness (dsh) 为例——一个“一切皆插件”的开源 agent harness，讲清三件事：它是什么（概念与循环）、怎么装起来（三条路径 + API Key）、怎么安全地用（权限与数据红线）。

学习目标

1. 说清代码补全、对话式编程与 Agent 编程的区别，描述一次 Agent 循环五个动作；
2. 用“五件套”（模型、工具、上下文、权限、会话）解释一个 harness 的组成；
3. 独立完成 dsh 的三条安装路径之一（npx 免安装 / 全局 CLI / 桌面版源码构建）；
4. 完成 DeepSeek API Key 的注册、实名、充值、创建与填写，并估算一次任务的花费；
5. 会用界面与斜杠命令（/plan、/compact、/goal、/model、/permission、/export 等）控制会话；
6. 说出权限三档、审批机制与数据红线，知道什么数据不能交给云端模型。

先修要求与运行环境

- 先修：第 0 章（终端、Python 环境）与第 1 章 NumPy（本部分案例会用 axis、广播、形状判据）；
- 运行环境：Node.js ^22.19.0 || >= 24.0.0 (npm 与 npx 随它一起安装)、一个 DeepSeek 开放平台账号与 API Key；
- 详细安装步骤见本章 02 节；无 Key 时也能完成理论、审查类练习与离线替代路径。

本章结构

小节	内容	状态
01 AI 编程与 Agent 基础	三代工具、harness 五件套、循环、上下文、人机分工、失败模式	已交付
02 安装与配置	Node.js 与 npm、三条安装路径、API Key 申请与付费、价格快照、故障排查	已交付
03 界面与斜杠命令	Web 界面导览、斜杠命令速查与示例（本地执行 vs 模型对话）	已交付
04 权限、安全与数据红线	权限三档、审批实践、数据红线与合规	已交付

资源导航

资源	位置	说明
理论篇	01-AI 编程与 Agent 基础.md	概念、循环、上下文、权限、失败模式

资源	位置	说明
安装与配置	02-安装与配置.md	三条路径、API Key 与付费、成本估算、故障排查
参考资料	references.md	官方文档、平台入口、通识材料
教学说明	teaching.md	教师用：课时、演示脚本、常见坑
练习	../11-agent-cases/exercises/README.md	全部分练习统一入口（quiz + assignment）
上机	../11-agent-cases/lab/README.md	全部分上机入口（离线可跑 + 联网任务）

版本与时效

本章内容与 dsh 版本强相关，因此有三层防护：

- 1. 版本单一事实源：docs/build/dsh_version.yaml（基线版本、核对日期、上游链接、需复核文件清单）；
- 2. 时效标记：正文中易变段落以“时效内容（基线 vX / 日期）”开头，便于批量复核；
- 3. 双周巡检：docs/build/check_dsh_release.py 与 .github/workflows/dsh-release-watch.yml 在官方发新版时提醒，流程见 docs/build/dsh_更新 SOP.md。

常见问题

问题	处理
没有 API Key 能学吗？	能：本章理论、第 10 章方法、第 11 章全部审查与离线练习都不需要 Key
一定要用 DeepSeek 吗？	教材以 DeepSeek Harness + DeepSeek 模型为主线；dsh 也支持自定义兼容端点
会不会很贵？	练习任务用 deepseek-flash，一次通常几分钱；估算方法见 02 节

01 AI 编程与 Agent 基础

本节回答三个问题：Agent 编程与“AI 帮我写代码”到底差在哪？一个 Agent 内部做了什么？哪些事必须自己管？阅读约 25 分钟；配套图 4 张，可直接运行的代码 1 段。

本节目标

- 1. 说清**代码补全**、**对话式编程**、**Agent 编程**三者的差别，并解释“闭环”为什么是关键。
- 2. 用**五件套**描述一个 agent harness（智能体框架）：模型、工具、上下文、权限、会话。
- 3. 复述一次 **Agent 循环** 的五个动作，并能把一个小任务拆成循环。
- 4. 解释**上下文窗口与压缩**，知道怎么“省上下文”。
- 5. 说出**权限三档**与审批机制，记住数据红线在哪里。
- 6. 识别**四种典型失败模式**，并给出至少一条对策。

先修

- 完成第 0 章的环境安装：会用终端（PowerShell / Bash）、会运行 python 脚本；
- 学完第 1 章 NumPy：本节的演示案例用到 axis 与广播；
- 不需要任何机器学习或大模型背景。

术语速查（中英对照）

后续章节会反复用到这些词，先混个脸熟即可：

术语	英文	一句话解释
智能体	Agent	能自己调用工具、执行多步任务来完成目标的 AI 程序
智能体框架	Agent harness	把模型、工具、上下文、权限、会话组装起来跑循环的软件
工具调用	Tool call	模型请求执行一个动作（读文件、跑命令、搜索）并拿到结果
上下文	Context	模型这一次能看到的全部内容：指令、历史、文件片段、工具输出
压缩	Compaction	上下文快满时，把过程压缩成“目标 + 结论 + 证据”
权限	Permission	决定 Agent 能做什么、哪些动作需要你点头
沙箱	Sandbox	限制 Agent 只能访问指定范围的机制
技能	Skill	预先写好的领域工作流，按需加载进上下文
子代理	Subagent	由主代理派生、专门做一件事的独立 Agent
计划模式	Plan mode	先产出计划、经你确认后再动手的工作方式
会话	Session	一次持续的对话上下文，可保存、继续、回看
工作区	Workspace	Agent 默认能读写的项目目录



Figure 1: 三代编程助手对比

正文

1. 三代编程助手：从“猜下一行”到“自己动手”

	代码补全	对话式编程	Agent 编程
你给它	正在写的这一行	报错、代码片段、问题	目标 + 约束 + 验收标准
它给你	猜出的下一行	代码与解释	读文件、跑命令、改代码、再验证的 完整过程
你负责	想清楚、跑通、验证	复制粘贴、串起来、验证	定义问题、定验收、做终审
出错时	你改	你改	它自己看报错再改（直到你设的验收标准满足）

三者的分水岭是**闭环**：前两代的输出是“文本”，你拿到文本后还要自己去运行、观察、判断、再回来描述问题；Agent 的输出是“完成的动作”——它有眼睛（读文件、看报错）和手（改文件、跑命令），能在这个循环里自我修正。

一句话记住：**补全帮你写一行，对话帮你写一段，Agent 帮你把事情做完——但“做对”的定义仍然由你给出。**

2. harness：模型之外的四件事

很多人以为“用了同一个模型，效果应该差不多”。实际上，同一个模型在不同 harness 里的表现可以差很远，因为决定体验的是另外四件事：

组成	作用	少了它会怎样
模型 LLM	推理与决策	没有大脑，什么也做不了
工具 Tools	读写文件、跑命令、搜索	只能说不能做，等于聊天机器人
上下文 Context	把项目指令、历史、文件内容喂给模型	它不知道你的项目规范，反复问同样的问题
权限 Permission	决定能不能写、要不要审批	要么什么都不敢做，要么风险失控

harness（智能体框架）由五件事组成

模型只是其中一件；差距往往在其余四件



Figure 2: harness 的组成

- 判：按你给的验收标准（形状必须是 (4,)、数值与手算一致）给出结论与修复。
- 关键认识：**循环的终止条件由你定义**。没有验收标准，Agent 就会以“看起来能跑”作为终点。

4. 上下文：Agent 的短期记忆

上下文窗口再大也会满。一次长任务里，系统提示、对话历史、读入的文件、工具输出会迅速堆积；快满时 harness 会做**压缩 (compaction)**，把过程浓缩成目标、结论与关键证据。

三条实用建议：

- 给最小可复现**：一段 20 行的复现脚本，比 2000 行项目源码更有用。
- 让它自己读文件**：说“读 src/clean.py 里 load_data 函数”比把代码整段贴进对话更省上下文，也更准确。
- 长任务主动收口**：阶段结束时让它总结“已确认的结论 + 未解决的问题 + 下一步”，这既是压缩，也是你检查方向的时机。

案例卡 1：一个“看起来都对”的错误——axis 用反了 目标：认识“静默数值错误”——代码能跑、数字看着合理，但结论是错的。

代码

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68]], dtype=float)
```

一次 Agent 循环的解剖

未达标就继续下一轮，达标才交付；必要时请人介入

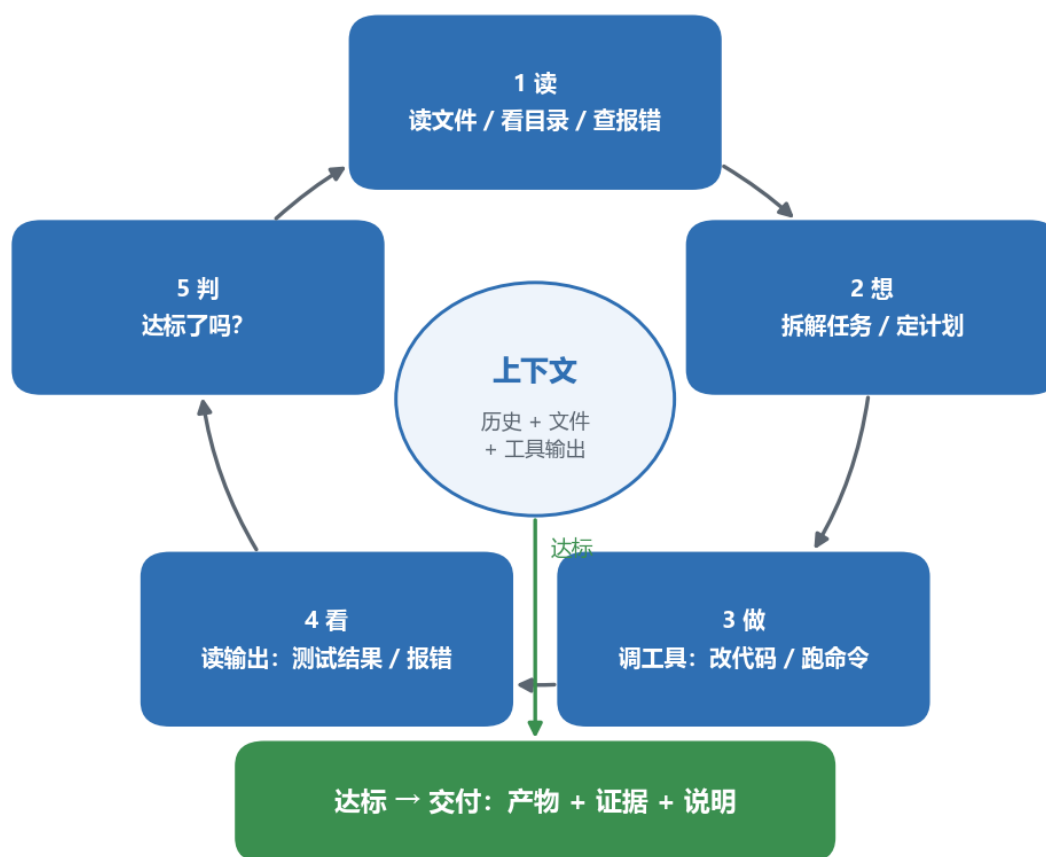


Figure 3: 一次 Agent 循环



常见错误

- 1. 只把结果打印出来看一眼“像不像”，不用形状与守恒量检查；
- 2. 把 axis=0 记成“对每一行”（其实是沿行方向压缩，得到每列的值）；
- 3. 让 Agent 修 bug 时只说“结果不对”，不说**期望的形状与判据**，它只能猜。

拓展

- 把 scores 换成 (3, 4, 5) 的三维数组，先说出 mean(axis=1) 的形状，再运行验证；
- 让 Agent 写一个 3 行的断言函数，把“形状必须是 (4,)”固化成测试；
- 思考：如果这份数据是某次实验的唯一结果，这个错误会怎样影响结论？

5. 人机分工：什么交给它，什么必须自己管

任务类型	建议	原因
样板代码、格式转换、批量重命名	交给 Agent	规则明确、易验证
写测试、补边界条件、重构	交给 Agent（你审 diff）	有客观判据
查文档、读陌生代码库、写注释	交给 Agent	信息检索型工作
问题定义与建模	人主导	决定“做什么”，Agent 无法替你决定
验收标准与数据判据	人主导	决定“什么叫做对”
结论与解释	人主导 （Agent 只能起草）	涉及因果、局限与责任

三条铁律：**问题定义归人、验收标准归人、结论归人**。Agent 可以在这三件事上给建议，但不能替你签字。

6. 权限与安全：让 Agent 在笼子里干活

主流 harness 都提供分档权限，常见三档：

档位	含义	适用场景
read-only	只能读，不能改	第一次接触陌生项目、只想让它解释代码
workspace-write	只能改工作区内的文件	日常开发、课程作业（ 推荐默认 ）
danger-full-access	全盘可写、命令直通	受控环境下的高级用法，慎用

配套机制是**审批**：涉及写文件、跑命令、装依赖时，界面会先问你；你的每一次允许或拒绝都会写进会话记录，可回看、可追责。

数据红线（请直接记住）：

- 不要把隐私数据（身份证、手机号、成绩单、医疗记录）贴进对话；
- 不要把涉密或未发表的数据交给云端模型；
- 不要把别人的 API Key、密码、令牌放进提示词或代码；
- 处理真实数据前，先想清楚“这份数据能不能离开这台机器”。

7. 四种典型失败模式与对策



Figure 5: 权限档位与审批

失败模式	长什么样	对策
幻觉 API 静默数值错误	编出不存在的函数、参数、配置项 能跑、数字合理，其实方向或单位错了	要求它运行验证；自己查一眼官方文档 事先给判据：形状、量纲、守恒量、解析解对照
过度自信的结论	把相关说成因果、不给不确定性	要求给证据、给区间、给反例，结论自己写
改坏别处	顺手重构、动了不该动的文件	小步提交、看清 diff、跑测试再接受

这四种失败都不是”模型不够聪明”，而是**缺验证**。第 03 节会给出一个固定的验收 workflow。

8. 上手体验（5 分钟）

如果你已经按第 02 节装好了 DeepSeek Harness，现在就可以试一次。任务：让它解释本节的 axis 案例。
我在做课程第 1 章的练习题，这段代码想算” 每个学生的平均分”，但我不确定对不对：

```
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)
by_exam = scores.mean(axis=0)
```

请你：

- 1) 指出这段代码算出来的到底是什么（给出形状）；
- 2) 写出正确写法，并给出一个最小复现脚本；
- 3) 加一条断言，保证结果形状符合” 每个学生” 的意图。

不要修改我工作区里的其他文件。

观察三个点：

1. 它有没有**先读文件、先运行**，还是直接给答案；
2. 它给出的断言是不是可执行、可验证；
3. 它有没有解释“为什么 axis=0 得到的是每列”，而不是只给结论。

本节小结

1. 三代工具的分水岭是**闭环**：能否自己执行、观察、修正。
2. harness = 模型 + 工具 + 上下文 + 权限 + 会话，后四件决定体验。
3. 循环五步：读、想、做、看、判；**终止条件由你的验收标准定义**。
4. 上下文是最稀缺资源：给最小复现、让它自己读文件、长任务主动收口。
5. 权限三档 + 审批 + 数据红线；问题定义、验收标准、结论永远归人。

思考题

1. 同一段代码，让“对话式助手”和“Agent”分别修一个 bug，你能观察到哪些行为差异？
2. 为什么“代码能跑通”不能作为验收标准？请举一个本课程里的例子。
3. 如果上下文快满了，你会保留哪三类信息？
4. 什么情况下应该主动把权限降到 read-only？
5. 同学说“Agent 写的报告结论和我们实验一致，所以没问题”。请指出这句话的逻辑漏洞。

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 1-4 题（概念题）；
- 延伸：DeepSeek Harness 官方仓库与文档（见 references.md）；
- 下一节：02-安装与配置.md（三条安装路径 + API Key 申请与付费）。

02 安装与配置

时效内容（基线 v0.1.5-rc.2 / 2026-09-19）：本节涉及版本号、命令、界面与价格，是全书最容易过期的部分。遇到不一致时以官方文档为准；更新流程见 docs/build/dsh_更新 SOP.md。

本节目标

- 1. 按自己的场景选对安装路径：**免安装 Web 版 / 全局 CLI 版 / 桌面版**。
- 2. 独立完成 **API Key** 的注册、实名、充值、创建与填入。
- 3. 配置模型并跑通**第一个任务**，知道怎么验收。
- 4. 说清 dsh 的**版本状态**，会自查版本与升级。
- 5. 会用价格表**估算一次任务的花费**。

先修

- 第 0 章的终端使用与 Python 环境；
- 会看命令行输出、会复制粘贴命令即可，不需要前端或运维经验。

0. 安装前的准备（先读 2 分钟）

0.1 dsh 的版本状态与时效

DeepSeek Harness (dsh) 目前处于官方所说的 **开发者预览 (Developer Preview)** 阶段，迭代很快：

- **教材基线**：0.1.5-rc.2 (npm 的 latest 标签，2026-09-10 发布；教材核对日期 2026-09-19)；
- **官方最新**：撰写时 GitHub Releases 上所有 tag 都标记为 prerelease，没有传统意义的“稳定版”；
- 也就是说：**npm latest 就是官方当前推荐的安装版本**，名字里的 rc 不代表“不能用于教学”。

三条自查命令（建议收藏）：

```
dsh --version           # 本机已装版本
npm view @deepseek-ai/dsh dist-tags  # 官方各渠道最新版本 (latest / next / alpha)
npm view @deepseek-ai/dsh version    # latest 渠道的具体版本号
```

官方入口（完整清单见 references.md）：

- 仓库：<https://github.com/deepseek-ai/deepseek-harness>
- 文档站：<https://deepseek-harness.github.io/deepseek-harness/>
- 版本记录：仓库 Releases 页（每个版本都有中文更新说明）

0.2 Node.js、npm、npx 是什么

dsh 本身是用 JavaScript / TypeScript 写的，所以需要有一个 JavaScript 运行时；它的安装包发布在 npm 仓库里，所以还需要一个包管理器。三者关系一句话：

Node.js 提供运行环境 → npm 负责从仓库取包、装包 → dsh 是 npm 上的一个包。

名词	全称	作用	本节里第一次用到
Node.js	——	让 JavaScript 脱离浏览器运行的运行时	运行 dsh
npm	Node Package Manager	Node 的官方包管理器：装包、升级、卸载	npm i -g @deepseek-ai/dsh

名词	全称	作用	本节里第一次用到
npx	npm exec	npm 自带的“临时执行器”：不永久安装，直接下载并运行	npx @deepseek-ai/dsh web

关键点：**安装 Node.js 时会自动装上 npm 与 npx**，不需要（也不应该）单独去装 npm。网上“先装 node 再装 npm”的教程是过时说法。

0.3 安装 Node.js（三平台 + 两种风格）

版本要求：^22.19.0 或 >= 24.0.0。推荐装 **Node 24 LTS**（当前 LTS 代号 Krypton, 24.x）；**Node 22 LTS**（Jod, 22.19 以上）也满足要求。

平台	方式	步骤 / 命令	适合谁
Windows	官网安装包（推荐）	打开 nodejs.org → 下载 LTS 版.msi → 双击安装（保持默认勾选 Add to PATH）→ 重开终端	绝大多数同学
Windows	winget	winget install OpenJS.NodeJS.LTS	已用 winget 管理软件的同学
Windows / macOS	版本管理器 fnm	winget install Schniz.fnm 或 brew install fnm, 再执行 fnm install 24 与 fnm use 24	需要多个 Node 版本共存
macOS	Homebrew	brew install node@24	已装 Homebrew
macOS / Linux	nvm	按 nvm 官方 README 提供的脚本安装，然后 nvm install --lts	服务器、多版本场景
Linux	发行版包管理器 / NodeSource	apt、dnf 等； 注意仓库里的版本可能偏旧 ，装完务必按 0.4 自查	有 root 权限的服务器

两条经验：

- 1. 装完**一定要重开终端**（环境变量 PATH 才会生效），否则会一直提示找不到命令；
- 2. 有版本管理器时优先用它——将来 dsh 要求更高版本时，一条命令就能切换。

0.4 装完自查

```
node -v           # 期望 v24.x 或 v22.19 以上
npm -v            # 能打印版本号即可
npx --version     # npx 随 npm 一起安装
npm config get registry # 默认 https://registry.npmjs.org/
```

如果 npm config get registry 显示的是公司内网地址，说明这台机器配置了私有仓库，安装 dsh 前请先确认该仓库能拉到公开包（否则改用个人电脑，或按管理员要求配置代理）。

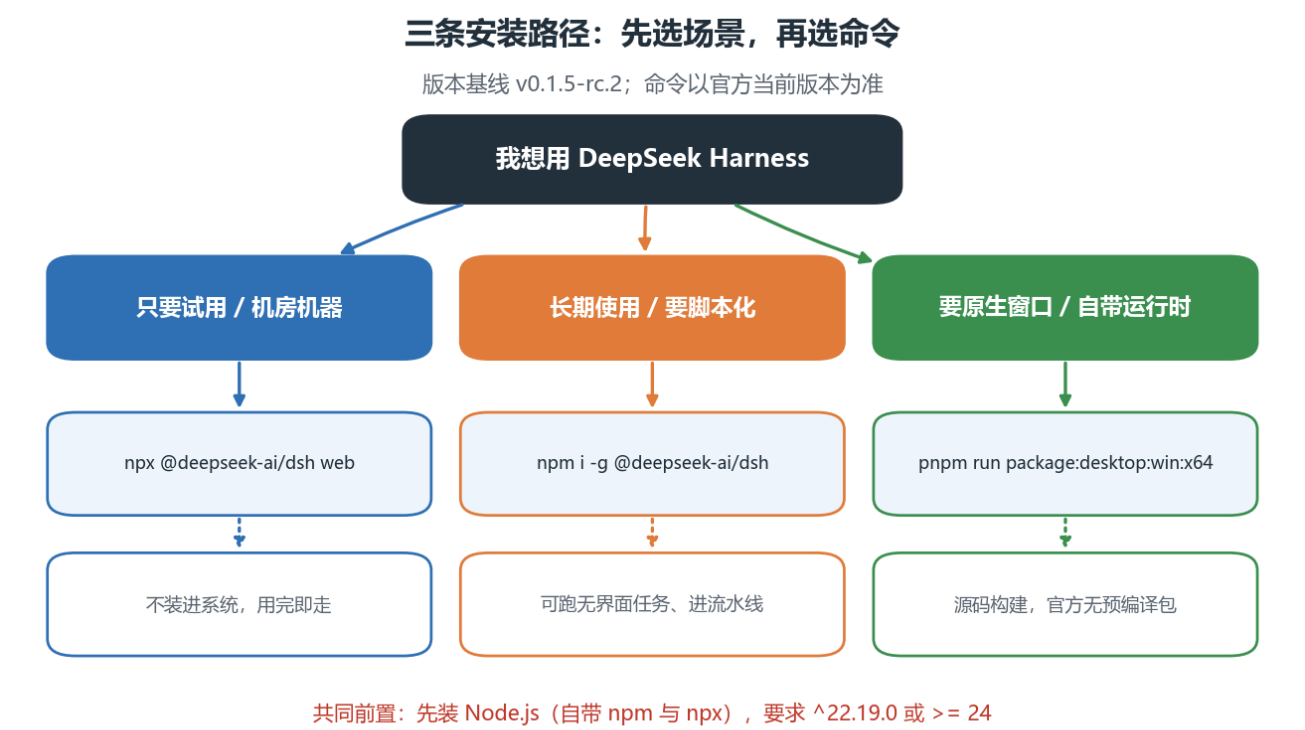


Figure 1: 三条安装路径

0.5 装不上怎么办

现象	可能原因	处理
node 不是内部或外部命令 / command not found	没装成功，或 PATH 未生效	重开终端；仍不行则重装（Windows 安装时保持勾选 Add to PATH）
npm 报 EACCES / EPERM 权限错误	用管理员或 root 装了全局包	不要用 sudo / 管理员装全局包；用普通用户重装 Node 后再试
下载很慢或超时	网络问题	临时换用国内镜像（命令见下方代码块）；或按校网要求配置代理
版本太低（例如 v20）	系统包管理器里的版本旧	用 fnm / nvm 安装 24 LTS，或从官网重装
公司电脑装不了	安装策略限制	用官网 zip 免安装版解压后手动加 PATH，或改用学校机器
卸载后旧版本还在	多个 Node 共存	先查来源：where node / which -a node，再清理

1. 三条路径怎么选

路径	适合谁	需要准备	升级方式
A 免安装 Web 版	第一次用、机房机器、不想改系统	Node.js	每次运行自动取最新（也可锁定版本）

路径	适合谁	需要准备	升级方式
B 全局 CLI 版	长期使用、要写脚本、要无界面跑任务	Node.js + npm	npm i -g @deepseek-ai/dsh@latest
C 桌面版	想要原生窗口与自带运行时	Node.js + pnpm + 源码构建	重新拉源码构建

共同前置只有一个：**Node.js** (npm 与 npx 随它一起安装，见 0.2 与 0.3)。官方仓库要求的版本是 ^22.19.0 或 >= 24.0.0。

```
node -v      # 例如 v24.19.0
npm -v       # 例如 11.x
```

如果 Node 版本过低，先升级：官网下载 LTS 版，或用 fnm / nvm 等版本管理器（第 0 章介绍过管理多个工具链的思路）。

提示：npm 下载慢时，可以临时换用国内镜像；企业网络或校园网如有代理，请按网络管理员要求配置。

```
npm config set registry https://registry.npmirror.com # 国内镜像（可选）
```

2. 路径 A：免安装 Web 版（推荐先试）

不安装任何全局命令，直接用 npx 拉起浏览器界面：

```
npx @deepseek-ai/dsh web
```

会发生什么：

- 1. 首次运行会下载依赖包（视网络情况可能需要几分钟），请耐心等待；
- 2. 服务默认在 http://127.0.0.1:3080 启动，本机运行时会自动打开浏览器；
- 3. **启动命令时所在的目录，就是默认工作区**，建议在一个专门的项目目录里执行；
- 4. 关闭：回到终端按 Ctrl+C。

常用参数：

```
npx @deepseek-ai/dsh web --port 8080      # 换端口（被占用时很有用）
npx @deepseek-ai/dsh web --no-open         # 只启动服务，不开浏览器
npx @deepseek-ai/dsh web --help           # 查看当前版本支持的全部参数
```

说明：如果通过 SSH 在服务器上启动，终端只会打印地址，需要你在本地做端口转发（进阶用法，本节不展开）。

3. 路径 B：全局 CLI 版（长期使用）

装一次，长期用：

```
npm i -g @deepseek-ai/dsh      # 全局安装
dsh --version                  # 确认版本
dsh web                        # 等价于 npx 的浏览器界面
```

本地版真正多出来的能力是**无界面（headless）任务**——让 Agent 跑完一件事直接打印结果，适合写进脚本：

```
dsh --profile headless "统计当前目录下所有 Python 文件的行数，输出一个表格"
```

其他自查入口：

```
dsh --help           # 启动器自身的参数
dsh web --help       # Web 应用的参数
dsh --dump-config    # 查看合并后的配置树（不启动）
```

首次运行会在用户目录下创建配置与数据目录（Windows 为 C:/Users/你的用户名/.dsh，macOS 与 Linux 为 ~/.dsh），里面包括设置、会话记录与凭据文件。**凭据文件不要提交到 git，也不要截图外发。**

升级与卸载：

```
npm i -g @deepseek-ai/dsh@latest    # 升级到最新
npm rm -g @deepseek-ai/dsh          # 卸载
```

对比项	npx 免安装	全局 CLI	headless
是否改系统	否	是（全局包）	是
界面	浏览器	浏览器	无（打印结果）
适合	试用、机房	日常使用	批处理、自动化
升级	每次运行自动解析	手动升级	跟随全局

4. 路径 C：桌面版（进阶，官方暂无预编译安装包）

先说结论：**官方目前没有发布桌面端安装包**，桌面端需要从源码构建，工程量偏大。课堂上了解它的定位与取舍即可，不要求真的构建。

桌面端是什么：它是完整 Web 应用外面的一层 Electron 壳——同一套界面与能力，多了原生窗口、菜单与系统通知，并**自带 Python / Node.js / pnpm 运行时**（对不想自己配环境的用户友好）。它默认使用 19387 端口，与 Web 版的 3080 分开。

源码构建（需要 Node 与 pnpm，命令以仓库当前版本为准）：

```
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm run package:desktop:win:x64      # Windows
# macOS Apple 芯片: pnpm run package:desktop:mac:arm64
# macOS Intel: pnpm run package:desktop:mac:x64
```

只想先看看界面，可以用开发模式启动：pnpm run dev:desktop。

社区项目：有第三方开发者为 Windows 打包了桌面版分发（例如 GitHub 上的 DeepSeek-Harness-Desktop）。它**不是官方发布**，无法审计构建过程，安装前请自行评估供应链风险；课程不把它作为推荐路径。

5. 配置模型与 API Key（重点）

5.1 为什么需要 API Key

dsh 本身是免费开源软件，但它在运行时需要调用大模型，而模型调用按用量计费。因此你需要一个 DeepSeek 开放平台账号与 API Key——可以理解为“给 Agent 用的加油卡”。

5.2 五步拿到 API Key

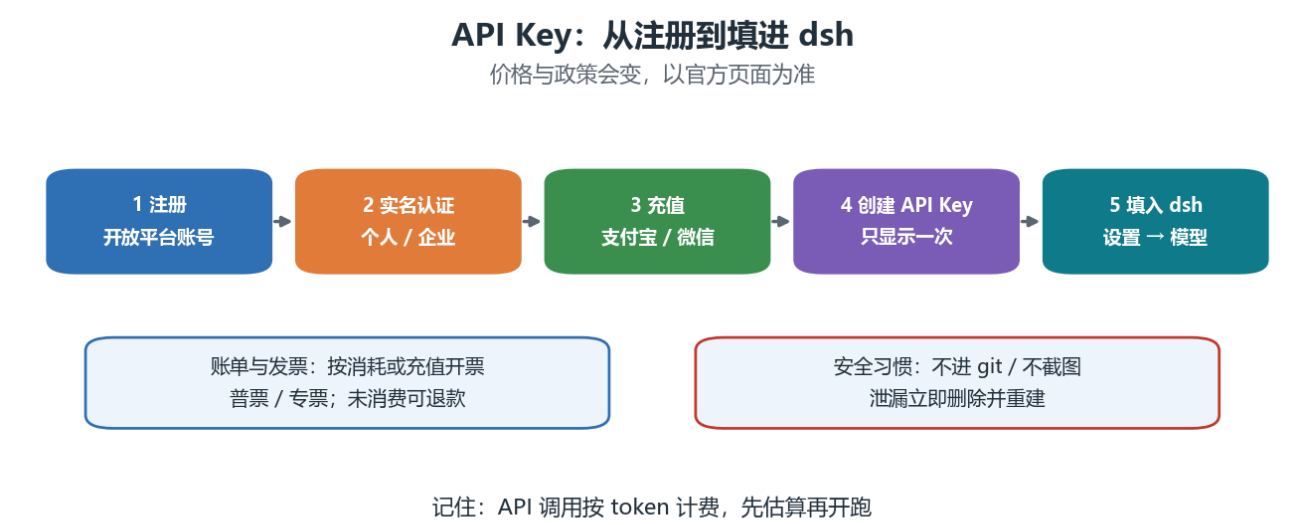


Figure 2: API Key 申请流程

步骤	在哪里做	关键细节
1 注册	platform.deepseek.com	支持手机号 / 邮箱 / Google 登录
2 实名认证	平台「实名认证」	分个人认证与企业认证； 决定后面发票抬头
3 充值	平台「充值」页	支付宝 / 微信在线充值；对公汇款仅企业用户
4 创建 API Key	平台「API keys」页	Key 只显示一次 ，当场复制保存
5 填入 dsh	Web 界面「设置 → 模型 → DeepSeek」	粘贴 Key 并保存，立即生效，无需重启

图：设置 → 模型。把 API Key 粘贴到 DeepSeek 卡片的输入框并保存（截图：基线 v0.1.5-rc.2 / 2026-09-19）。

关于余额与发票（来自官方常见问题，2026-09 核对）：

- **充值余额永久有效**，未消费金额支持退款；
- 若同时存在赠送余额与充值余额，优先扣减赠送余额；
- 发票：在「账单 → 发票管理」申请，可按**消耗金额**或**充值金额**开票；支持增值税普通发票与专用发票；个人认证可开个人或公司抬头，企业认证只能开企业认证主体的抬头；已开票金额需先作废才能退款。

5.3 价格快照（2026-09-18 公布口径）

价格以“ 每百万 tokens” 计，且**分高峰与空闲两档**（空闲时段为高峰的一半）。

模型	输入（缓存命中）	输入（未命中）	输出	备注
deepseek-flash	空闲 0.02 元 / 高峰 0.04 元	空闲 1 元 / 高峰 2 元	空闲 4 元 / 高峰 8 元	支持图片；上下文 1M
deepseek-v4-pro	空闲 0.15 元 / 高峰 0.30 元	空闲 4.5 元 / 高峰 9.0 元	空闲 13.5 元 / 高峰 27.0 元	纯文本

高峰时段：北京时间周一至周五 9:00–12:00、14:00–18:00；其余时间为空闲时段（半价）。价格与政策会调整，**以官方定价页为准**。课程建议：练习与作业用 deepseek-flash；把重跑安排在空闲时段，能省一半。

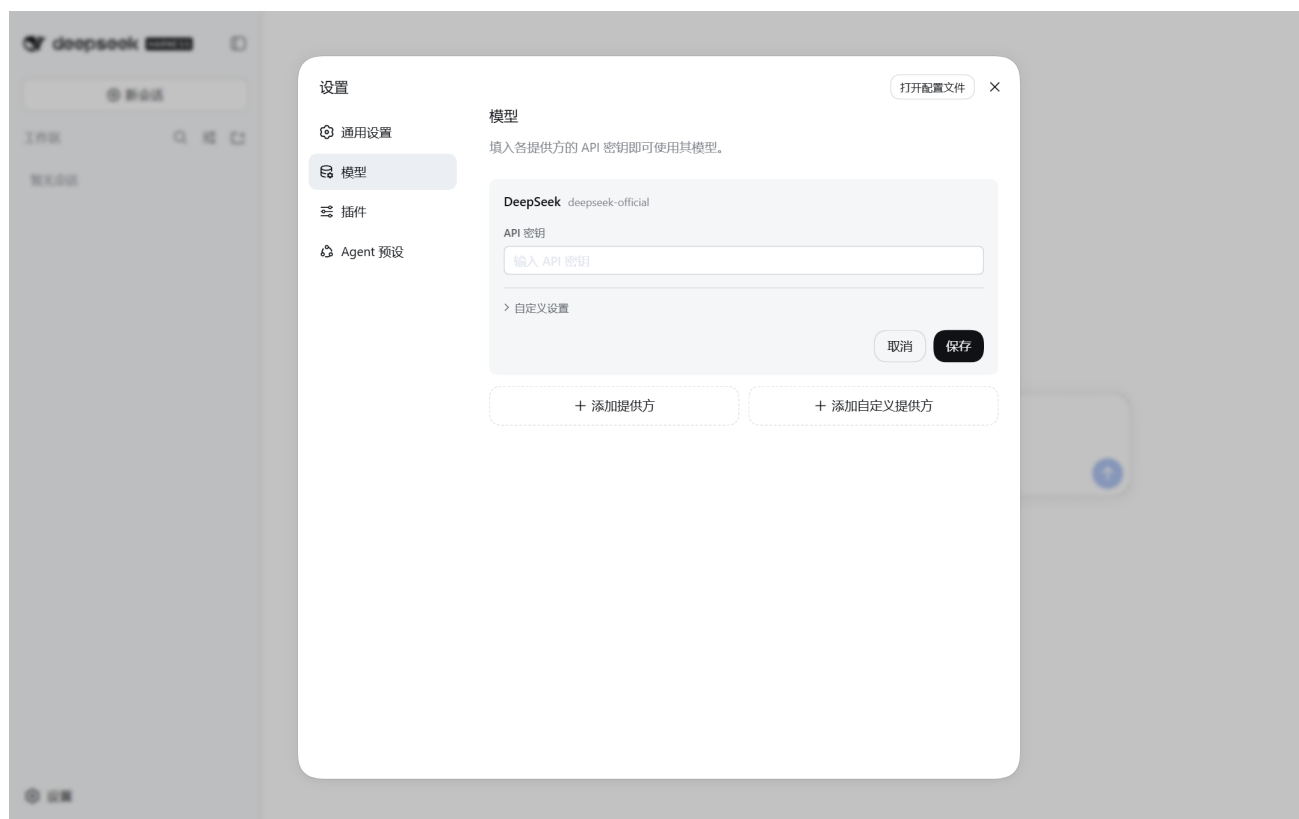


Figure 3: 设置 → 模型：填入 API Key

flash off-peak: 0.0812 CNY
flash peak : 0.1624 CNY
v4-pro off-peak: 0.3015 CNY

结论：一次“读代码 + 改代码 + 跑测试 + 写说明”的中等任务，用 deepseek-flash 大约几分钱到几角钱；换成 deepseek-v4-pro 会贵 3 到 4 倍。

6. 第一个任务（5 分钟验收）

1. 在浏览器界面点「选择工作区」，添加你的课程项目目录；
2. 新建会话，把下面这段提示词发给它：

请阅读当前工作区，回答三个问题：

- 1) 这个项目是做什么的？列出主要目录及用途；
- 2) 有哪些 Python 脚本？各自的作用是什么？
- 3) 如果我要新增一个数据分析脚本，应该放在哪里、遵循什么命名习惯？

只读不改：不要修改任何文件。

3. 观察它调用了哪些工具（读文件、列目录、搜索），以及它给出的结论有没有**证据**（引用了具体文件与行）。

验收清单：

- 它确实读了文件，而不是凭空描述；
- 它引用的文件名与目录结构能在你的项目里对上；
- 整个过程没有修改任何文件（因为你在提示词里要求了只读）。

7. 常见故障排查

现象	可能原因	处理
npm: command not found	没装 Node.js	安装 Node LTS 后重开终端
npx 卡住不动	网络慢或代理	换网络、配置镜像或代理；耐心等待首次下载
端口 3080 被占用	已有实例在跑	用 -port 8080 换端口，或先关闭旧实例
浏览器打不开页面	服务没起来或防火墙拦截	看终端是否打印了地址、有没有报错
输入框灰色不可用	没有选择工作区	先「选择工作区」并选中
模型报鉴权错误	Key 填错或已被删除	重新在「设置 → 模型」粘贴 Key
dsh 全局命令找不到	全局 bin 目录不在 PATH	重开终端；或改回用 npx
PowerShell 执行策略报错	脚本被策略拦截	改用 npx 直接运行，或按学校要求调整策略
任务跑很久没结果	任务过大或网络慢	缩小任务范围，先给最小复现

8. 升级与版本自查

```
npm view @deepseek-ai/dsh dist-tags      # 看官方最新
dsh --version                             # 看本机版本
npm i -g @deepseek-ai/dsh@latest          # 升级
```

升级前建议读一眼 Releases 页的更新说明（官方每个版本都有中文条目），重点关注三类变化：**命令与参数**、**界面位置**、**模型名称**。这三类正是本节内容的来源。

本部分的维护机制：docs/build/dsh_version.yaml 记录基线版本，docs/build/check_dsh_release.py 定期比对官方版本，双周自动提醒（见 docs/build/dsh_更新 SOP.md）。

本节小结

- 1. 三条路径：npx 免安装（先试）、全局 CLI（长期与脚本化）、桌面版（源码构建，进阶）。
- 2. 共同前置是 Node.js（^22.19.0 或 >= 24.0.0）。
- 3. API Key 五步：注册、实名、充值、创建、填入「设置 → 模型」。
- 4. 计费按 token、分高峰与空闲；练习用 deepseek-flash，重跑放空闲时段。
- 5. Key 与凭据文件是资产：不进 git、不截图、泄漏立即删除重建。

思考题

- 1. 同学说“我用 npx 跑起来了，所以不用装 Node”——这句话哪里不对？
- 2. 为什么教材不推荐学生用第三方打包的桌面版？请列出两条风险。
- 3. 同样的任务，在高峰时段与空闲时段跑，费用差多少？请用 5.4 的脚本算一个例子。
- 4. 如果 API Key 不小心提交到了 git 仓库，你应该按什么顺序处理？
- 5. 你想让 Agent 每天晚上自动跑一次数据检查，应该选哪条路径、用什么入口？

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 5–6 题；
- 上机：lab/lab.ipynb Part A（环境自检，可无网络完成）；
- 参考：官方 README、Web UI 指南、模型配置指南（见 references.md）。

03 界面与斜杠命令

本节解决“装好之后怎么用”：Web 界面在哪里、斜杠命令怎么用、哪些操作**不经过模型**。基线版本：dsh 0.1.5-rc.2（2026-09-10）；命令清单与菜单分组随版本变化，权威说明见官方文档站。

本节目标

- 1. 认识 Web 界面的主要区域：工作区、会话、设置（模型）、权限指示；
- 2. 理解斜杠命令与普通对话的**本质区别**：命令在本地执行，不会变成模型消息；
- 3. 掌握 8 条常用命令的语法与使用时机；
- 4. 知道哪些入口**没有**命令面（headless、SDK、ACP）。

先修

- 02 节：已经装好 dsh，并完成模型与 API Key 配置。

3.1 界面导览（Web 版）

区域	位置	作用
工作区选择	首屏 / 左侧	Agent 能读写的目录边界； 没选工作区时输入框不可用
会话区	中间	对话、工具调用与结果的流水账（这就是“证据链”）
输入框	底部	输入任务；点 + 按钮或输入斜杠打开同一个菜单
设置 → 模型	右上	选择模型、填 API Key、切换推理等级
权限指示	会话附近	当前权限档位；触发写文件或命令时会弹出审批

图：首次打开的界面。**没选工作区时输入框不可用**（截图：基线 v0.1.5-rc.2 / 2026-09-19）。

图：点「选择工作区」后添加你的项目目录。

输入框的 + 菜单与斜杠菜单是同一个（各版本措辞可能不同），分两段：

- **Add**：File（把工作区文件加入上下文）、Goal、Plan、Feedback；
- **Commands**：Compact、Permission、Model、Export。

3.2 斜杠命令是什么：不经过模型的控制面板

普通对话会作为消息进入模型上下文；**斜杠命令在本地直接执行**，命令本身与执行结果默认不会变成模型消息（因此也不消耗 token）。

命令行的语法固定：斜杠开头 + 小写名称（字母、数字、下划线、连字符）+ 空格 + 参数。名称后面的全部内容都是该命令自己的参数（各命令语法不同）。

重要限制：**命令面只在交互式界面（Web / CLI）存在**。用 headless 模式跑脚本、或通过 SDK / ACP 集成时没有这套命令，需要在提示词里用自然语言表达需求。

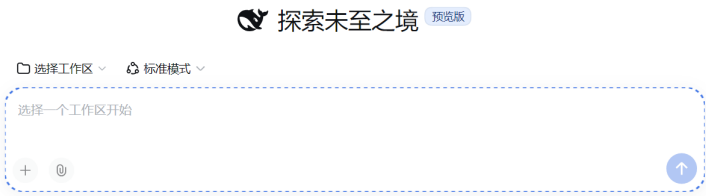
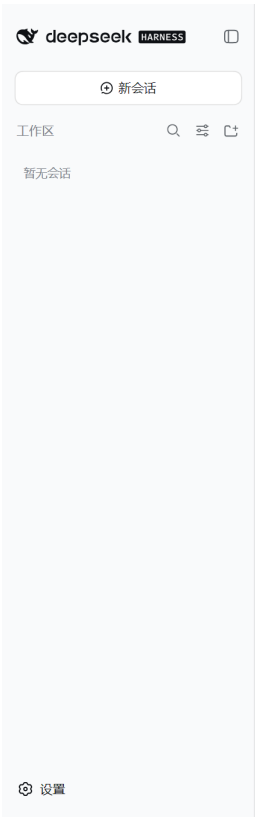


Figure 1: Web UI 首屏：选择工作区

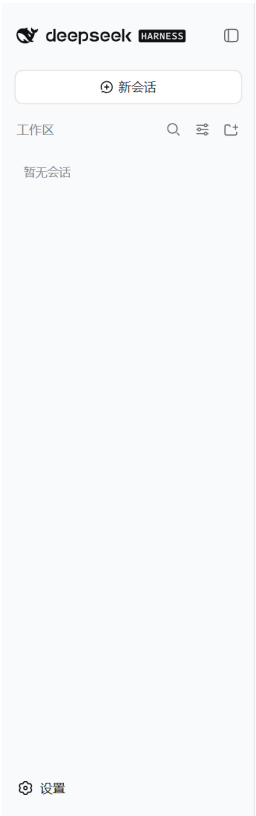


Figure 2: 选择工作区

3.3 速查表（基线 v0.1.5-rc.2）

命令	语法	作用	什么时候用
/compact	无参数	压缩较早的会话历史	会话很长、开始变慢或答非所问时
/plan	/plan [off message], 支持附件	进入或退出计划模式	多文件改动、动数据之前
/goal	/goal [目标 clear edit 目标 pause resume]	设定或查看长任务目标	跨多轮的长任务（论文复现、项目开发）
/feedback	/feedback 文本	记录对本次会话的反馈	发现工具不好用、想留改进线索
/export	无参数（仅 Web）	把会话日志打包成 ZIP 下载	交作业 、留档、复盘
/model	打开弹窗	切换模型与推理等级	想省钱（换 flash）或想更强推理（换 pro）
/permission	打开弹窗	切换权限档位	先只读审视，再放开写
/file	打开文件选择	把工作区文件加入上下文	想让 Agent 只看某个文件时

时效内容：命令名与菜单分组由插件提供，可能随版本增删。请用斜杠菜单查看当前版本的实际清单；升级后复核本表。

3.4 三个真实用法示例

示例 1：先计划、再立目标（大任务开场）

/plan 帮我用 sklearn 的 wine 数据做一个分类小项目，先给我计划，不要动手（看完计划、确认没问题后）
/goal 完成 wine 分类小项目：脚本 + 5 折交叉验证 + 报告

要点：/plan 只负责先看计划，/goal 负责记住目标，两者都不需要额外对话轮次去解释。

示例 2：会话太长，先压缩再继续

/compact
（然后继续提要求）请把刚才的 verify.py 补上数据泄漏检查

要点：压缩会丢失部分细节，**重要结论要写进工作区文件**（如 output/review.md），不要只留在对话里。

示例 3：换模型省钱 + 收紧权限 + 导出交作业

/model # 选择 deepseek-flash（练习阶段）
/permission # 切到 workspace-write（或更小的 read-only）
/export # 会话日志打包下载，作为作业附件

要点：/export 产出的 ZIP 是“AI 使用声明”的最强证据；把它和脚本、报告一起提交。

3.5 两个最容易踩的坑

- 1. **以为命令内容会进入模型上下文**：不会。想让 Agent 知道你的目标，要用自然语言再说一遍，或写进 AGENTS.md；
- 2. **把命令当成万能开关**：命令只改会话层的行为（上下文、权限、模型、目标），不改变你的验收标准——判据仍然由你写。

3.6 五分钟练习

1. 在工作区里新建一个空目录，用 `/plan` 让 Agent 给出“读取一个 CSV 并输出摘要”的计划；
2. 确认计划后，用 `/permission` 把权限调到 `read-only`，再让它执行——观察审批与拒绝的提示；
3. 用 `/compact` 压缩会话，检查它是否还记得你的目标（如果不记得，说明目标该写进文件）；
4. 用 `/export` 下载会话日志，打开看看里面记录了哪些工具调用。

本节小结

- 斜杠命令是本地控制面：不进模型上下文、不消耗 token；
- 8 条常用命令覆盖“上下文、计划、目标、模型、权限、导出”六件事；
- 命令面只在交互式界面存在，脚本化（headless）时要用自然语言；
- `/export` 让“提交会话记录”变成一条命令，直接服务于课程作业要求。

思考题

1. 为什么“命令不进入模型上下文”既是优点也可能是坑？请举一个具体场景。
2. 会话压缩后，哪三类信息最值得提前写进工作区文件？
3. 如果你要用 headless 模式每天自动跑一次数据检查，斜杠命令还能用吗？该怎么表达“先给计划”？

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 2 题（命令与场景匹配）；
- 延伸：官方文档站的 Web UI 指南与 CLI 参考（见 [references.md](#)）。

04 权限、安全与数据红线

本节回答“能放心让它干活吗”：权限三档怎么选、审批怎么用、哪些数据不能碰、出事怎么收场。基线版本：dsh 0.1.5-rc.2（2026-09-10）；权限与审批的具体界面随版本演进，以官方文档为准。

本节目标

- 1. 说清权限三档（read-only / workspace-write / danger-full-access）的差别与适用场景；
- 2. 掌握“从小到大放开”的实践顺序，并识别高风险动作清单；
- 3. 记住四条数据红线，知道课程作业与科研场景的合规要求；
- 4. 出事后能按固定顺序处置（止损 → 取证 → 修复 → 复盘）。

先修

- 01 节（失败模式与权限概念）、02 节（已装好并配好 Key）。

4.1 权限三档：默认最小，按需放开

档位	Agent 能做什么	什么时候用	风险
read-only	只读文件、列目录、跑只读命令	第一次接触陌生项目；让它解释代码	极低（唯一风险是信息被读进上下文）
workspace-write	只能改工作区内的文件	日常开发、课程作业（ 推荐默认 ）	中（误改工作区文件，可用 git 回滚）
danger-full-access	全盘可写、命令直通	受控的沙箱/容器；一次性脚本环境	高（可能改动系统文件、装软件、联网）

经验规则：能用 read-only 完成的，不要开到 write；能在工作区完成的，不要开 full-access。

4.2 从只读到放开的四步实践

- 1. 进入新项目：read-only，让它“讲一遍”项目结构与你关心的文件；
- 2. 确认它读懂了（能引用具体文件与行）：切 workspace-write，只做单文件小改动；
- 3. 每完成一步看一次 git diff：确认改动范围与预期一致；
- 4. 涉及下列**高风险动作**时，逐条确认（不要批量放行）：

高风险动作	为什么要确认
删除 / 覆盖文件、git reset --hard	不可逆，且可能破坏原始数据
批量重命名、批量替换	影响面大，容易波及其他章节/脚本
安装依赖、修改环境变量	改变本机环境，可能影响其他课程项目
联网上传、调用外部 API	数据可能离开本机
访问工作区之外的路径	越过本次任务的边界

4.3 审批怎么用：把“同意”变成决策

审批弹窗出现时，不要条件反射点“允许”。三个问题快速判断：

1. 这个动作**属于当前任务**吗？（不属于就拒绝）
2. 它**可逆**吗？不可逆就要求先备份或先只读验证；
3. 我能**看懂**它在做什么吗？看不懂就先让它解释，再决定。

除此之外，把“长期约定”写进 AGENTS.md（第 10 章 01 节），比每次审批更省事：例如“禁止修改 data/ 下的原始数据”“脚本一律输出到 output/”。

4.4 数据红线（四条，必须记住）

1. **个人隐私**：身份证号、手机号、成绩单、医疗记录、聊天记录——不贴进对话、不让她读取；
2. **涉密与未发表数据**：课题未发表数据、企业数据、竞赛内部数据——不交给云端模型；
3. **凭据**：API Key、密码、令牌、.credentials.yaml、.env——不写进提示词、不写进代码、不进 git；
4. **他人数据与版权**：未经许可不要上传/处理他人数据；教材与论文内容遵守许可。

判断口诀：“**这份数据能不能离开这台机器？**”答案不确定时，默认不交出；需要处理时，先做脱敏（去掉标识列）或用本地模型（第 12 章 02 节）。

4.5 API Key 与凭据的日常习惯

- Key 只保存在 dsh 的凭据文件里（02 节 5.2），**不要**复制到笔记、聊天、notebook 或脚本里；
- 不要把 .credentials.yaml、.env 截图；提交作业前用 git status 检查一遍；
- 一旦怀疑泄漏：立即到平台删除该 Key → 新建 Key → 更新 dsh 设置 → 检查账单用量；
- 课堂练习建议用**单独的低额度 Key**，与个人长期使用的 Key 分开。

4.6 出事之后的处置顺序（止损优先）

步骤	做什么	要点
1 止损	停止会话（关掉实例或按中断）	先别再让它跑命令
2 取证	导出会话（/export）、看 git status / git diff	会话日志是唯一完整记录
3 恢复	用 git 回滚、从备份恢复数据	原始数据优先，脚本其次
4 复盘	写下“哪一步放行错了”“下次加什么约定”	更新 AGENTS.md 与检查清单

4.7 小结与练习

小结：权限三档按需放开；高风险动作逐条确认；四条数据红线；出事按“止损 → 取证 → 恢复 → 复盘”处置。

练习（可离线完成）：

1. 给你的课程项目写一份 AGENTS.md 的“安全约定”段落（不超过 8 行）；
2. 列出你手上数据里**绝对不能**交给云端模型的三类，并说明理由；
3. 用第 11 章的 sample_agent_output/ 做一次“审批演练”：假设 Agent 请求删除 data/scores.csv，写出你的判断与回复。

思考题

1. 为什么“工作区边界”比“文件级权限”更实用？
2. 如果 Agent 建议把 scores.csv 里的 999 直接改成 100，你会怎么回复？请给出兼顾效率与安全的做法。

3. 团队协作时，如何让“权限约定”不依赖某个人记得？

延伸阅读

- 官方文档：权限与审批相关章节（见 [references.md](#)）；
- 下一章：第 10 章《Agent 工作流与验证方法》。

第 10 章

Agent workflow与验证方法

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

10. Agent 工作流与验证方法

本页是第二部分第 10 章的章首页。上一章回答“它是什么、怎么装”，本章回答“怎么干才靠谱”。

本章概览

Agent 编程的成败，很少取决于模型强弱，而取决于**方法与验证**：有没有把项目规范写进 AGENTS.md、复杂任务有没有先出计划、有没有把“什么叫做对”写成可执行的判据、关键结论有没有第二双眼睛。

本章把这套方法固化成可重复的操作流程，并在最后一节给出科研规范与学术诚信要求（AI 使用声明）。

学习目标

1. 会写最小可用的 AGENTS.md，把项目约定变成常驻指令；
2. 会用计划模式推进复杂任务，并用“小步执行 + 逐步验证”控制返工；
3. 会用**任务卡六件套**（目标、约束、验收标准、证据、复核、提交）描述任务；
4. 会为科学计算任务写出**可执行的验收标准**，并用断言把结论钉死；
5. 知道什么时候该派子代理、什么时候纯属浪费；
6. 说出 12 条常见误区，并按规范提交 AI 使用声明。

本章结构

小节	内容	状态
01 项目级指令与计划模式	AGENTS.md、计划先行、会话与上下文、权限约定固化	已交付
02 任务卡与验收标准库	任务卡六件套、验收标准正反例、第 1–8 章判据库、提示词清单	已交付
03 子代理、技能与工具	工具与技能、子代理判据、协作规矩与成本	已交付
04 常见误区与科研规范	12 个误区、AI 使用声明模板、可复现三件套、学术诚信	已交付
02 任务卡与验收标准库	六件套、“验收标准写法的正反例”、面向第 1–8 章的判据库	编写中
03 子代理、技能与工具	角色分工、成本、技能沉淀	编写中
04 常见误区与科研规范	12 条误区、AI 使用声明模板、可复现三件套、学术诚信红线	编写中

资源导航

资源	位置	说明
本章正文	01 · 02 · 03 · 04	四节构成完整方法链
参考资料	references.md	官方文档与延伸阅读
教学说明	teaching.md	教师用：课时、案例拆解、评分要点
配套案例	../11-agent-cases/README.md	本章方法的落地演练

与前后章的关系

- 上一章（第 9 章）解决“是什么、怎么装、怎么安全用”；
- 本章解决“方法”；
- 下一章（第 11 章）用四个递进案例把方法跑一遍；
- 第 12 章解决“长期怎么维护”（版本、成本、协作、项目）。

01 项目级指令与计划模式

本节解决“怎么让 Agent 一开始就按你的规矩干活”：把约定写进 AGENTS.md、复杂任务先出计划、长任务管好上下文。权限与安全的完整实践见第 9 章 04 节；本节只讲“如何把权限约定固化下来”。

本节目标

1. 会写一份最小可用的 AGENTS.md，让 Agent 不用每次重复问规范；
2. 会用“计划先行 + 小步执行”推进复杂任务；
3. 会管理会话与上下文：一次一件事、阶段收口、换任务开新会话；
4. 会把权限与数据约定写进项目文件，而不是靠每次口头提醒。

先修

- 第 9 章 01-04 节（概念、安装、界面与斜杠命令、权限与数据红线）。

1.1 项目级指令：AGENTS.md

AGENTS.md 是放在项目里的“常驻说明”，Agent 每次工作都会先读它。它解决的是**重复解释**问题：与其每次都读 CSV 要指定 UTF-8、图存到 output/、随机种子固定 42，不如写进文件。

最小可用模板（复制到项目根目录即可）：

项目说明

本项目是《Python 科学计算》课程的数据分析练习。

环境

- Python 3.12, 依赖见 environment.yml (numpy / pandas / scipy / statsmodels / scikit-learn / matplotlib)
- 运行命令统一用 python < 脚本名 >

约定

- 读 CSV 一律显式指定编码 utf-8-sig, 并先打印列名
- 随机过程必须固定种子: np.random.default_rng(42)
- 输出文件放 output/, 不覆盖 data/ 里的原始数据
- 中文图必须设置字体: Microsoft YaHei 或 SimHei

安全 (对应第 9 章 04 节)

- 只允许改工作区内的文件；删除、批量替换、装依赖前必须先问我
- 不读取 .credentials.yaml / .env 等凭据文件，不把它们的内容写进任何输出

质量要求

- 每个脚本末尾要有 assert 自检；改动后必须运行一次并贴出输出
- 不确定的地方先问我，不要擅自扩大改动范围

三条经验：

1. **只写“会被违反”的约定**：写得越长越没人看，Agent 也一样；
2. **写清验收方式**：能让它自查的约定，比形容词有用；
3. **把 AGENTS.md 提交到 git**：它是项目的一部分，不是个人偏好。

1.2 计划先行：复杂任务先要计划

任务超过“改一个文件”的量级时，先让它进入**计划模式**（界面里的 /plan，见第 9 章 03 节），产出：要动哪些文件、分几步、每步怎么验证、有什么风险。你确认后再动手。

任务规模	建议做法
一行改动、明确的小修	直接做，人审 diff
单文件重构、补测试	先说明要做什么，再执行
多文件、跨目录、动数据	必须计划模式 + 你确认
涉及删除、覆盖、联网安装	计划 + 逐条确认

计划模式的价值不是“礼貌”，而是把**返工成本**从“改完发现方向错”降到“看计划发现方向错”。

1.3 会话与上下文管理

- **一次会话只做一件事**：上下文里塞太多目标，它就容易抓错重点；
- 长任务定期收口，让它输出“已确认的结论 / 未解决的问题 / 下一步”——这三行就是压缩后的上下文；
- 换任务就开新会话，把上一轮结论作为开场信息带过去；
- 会话过长时用 /compact，但**重要结论要落到文件里**（如 output/review.md）；
- 会话记录是**证据链**：提交作业时用 /export 导出或截图留档。

1.4 把权限与数据约定固化下来

审批是“事后同意”，AGENTS.md 是“事前约定”。两者配合：

场景	写在 AGENTS.md 里的约定
原始数据	只读，不修改、不重命名 data/ 下的文件
输出产物	一律写到 output/，文件名固定（便于验收）
危险动作	删除、批量替换、安装依赖、联网前必须先说明并获得同意
凭据	不读取、不打印、不写入任何凭据文件

本节小结

1. AGENTS.md 把“每次都要说的规范”变成项目文件，并纳入版本管理；
2. 复杂任务先计划、后执行、小步验证；
3. 会话一次一件事，阶段收口，重要结论落盘；
4. 权限与数据约定要写进文件，而不是靠记忆。

思考题

1. 为什么“只写会被违反的约定”比“写得越全越好”更有效？
2. 计划模式在什么情况下反而是负担？请举一个具体例子。
3. 如果一个约定已经写进 AGENTS.md，但 Agent 仍然违反了，你会怎么排查？

练习与延伸阅读

- 练习：第 11 章 `exercises/` 第 6 题（为你的项目写 `AGENTS.md`）；
- 下一节：[02-任务卡与验收标准库.md](#)。

02 任务卡与验收标准库

本节是全部分的核心方法：把任何任务写成**可验收**的任务卡，并为科学计算任务准备一套现成的判据库。

本节目标

- 1. 会用**任务卡六件套**描述任务；
- 2. 会把形容词式要求改写成**可执行断言**；
- 3. 能从“验收标准库”里为第 1–8 章的典型任务挑选判据；
- 4. 会用提示词七条清单把任务说清楚。

先修

- 第 9 章（Agent 循环与失败模式）、本章 01 节（AGENTS.md 与计划模式）。

2.1 任务卡六件套

要素	写什么	例
目标	一句话说清要达成什么	算出每个学生的平均分
约束	不许动什么、用什么环境	只新增 scores_fixed.py, 不动 data/
验收标准	可执行 的判据	长度 8、形状 (8,)、与手算一致
证据	要它给出什么证明	运行输出 + 断言结果
复核	人要检查哪几点	归一化是否逐列、排名是否带姓名
提交	交付形态	文件 + 说明 + 会话记录

2.2 验收标准的写法：从形容词到断言

差的写法	好的写法
结果要正确	平均分长度必须是 8，且与手算 [83.33, 65.0, ...] 一致
图画得好看点	图保存到 output/trend.png；x 轴为小时、y 轴为温度（摄氏度）；标题含“传感器温度”
检查有没有问题	用 assert 检查：行数守恒、缺失值策略已记录、归一化逐列
模型效果要好	5 折交叉验证最低分 ≥ 0.90 ，且预处理在 Pipeline 内

为什么必须可执行：只有可执行的判据才能被反复验证。同一段代码，两种 axis，只有一种能通过断言：

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)

def check_student_mean(m):
    m = np.asarray(m)
    assert m.shape == (4,), " 形状必须是 (4, ), 每个学生一个平均分"
    assert np.allclose(m, scores.mean(axis=1)), " 数值必须与手算一致"
    return True
```

```
for axis in (0, 1):
    try:
        check_student_mean(scores.mean(axis=axis))
        print("axis=%d -> 断言通过" % axis)
    except AssertionError as e:
        print("axis=%d -> 断言不通过: %s" % (axis, e))

axis=0 -> 断言不通过: 形状必须是 (4,)，每个学生一个平均分
axis=1 -> 断言通过
```

2.3 验收标准库（面向第 1–8 章）

写任务卡时，直接从下表挑判据，改成你的具体数值即可：

章节 / 任务类型	可执行判据（举例）
第 1 章 NumPy	形状与广播结果 (x.shape); 与手算/解析解对照 (np.allclose); 视图与拷贝 (np.shares_memory); 残差与条件数 (np.linalg.norm(A@x-b)、np.linalg.cond(A))
第 2 章 SymPy	符号结果数值代入验证 (expr.subs); 两式之差化简为 0 (simplify(a-b) == 0); 与数值方法交叉验证
第 3 章 SciPy	积分与解析解对照; 优化收敛信息 (res.success、res.fun 对比理论最优); 插值在节点处误差为 0; 检验 p 值与手算一致
第 4 章 pandas	行数守恒 (清洗前后差值 = 删除行数); 缺失策略留痕; 分组聚合与手算对照; 表格一致性 (assert_frame_equal)
第 5 章 Matplotlib	图保存路径与 dpi; 坐标轴标签与单位; 中文无乱码; 随机种子固定; 数据点数量与源数据一致
第 6 章 NetworkX	节点/边数与源数据一致; 连通性 (nx.is_connected); 最短路长度与手算对照; 社区划分的模块度
第 7 章 Statsmodels	系数符号与量级合理; p 值与置信区间; R² 与残差诊断 (Durbin-Watson); 重跑一致 (固定种子)
第 8 章 sklearn	交叉验证分数区间; Pipeline 结构检查 (防泄漏); 与基线对照 (DummyClassifier); 指标口径与混淆矩阵一致
通用工程判据	脚本可重复运行; 输出文件存在且非空; 断言全部通过; git diff 只涉及预期文件

用法：把“第 8 章那类任务”的判据**数字化**——例如“5 折最低分 ≥ 0.90 ” “行数守恒：11 $\rightarrow$ 10，删 1 行” “max(axis=0) 全为 1”。

2.4 提示词七条清单

- 1. 说清目标与判据，不要只说“帮我看看”；
- 2. 给最小复现：能跑的 20 行，胜过 2000 行项目；
- 3. 限定范围：明确“只改 X、不动 Y”；
- 4. 要求自证：让它跑一次并贴输出，而不是“我觉得对”；
- 5. 要求打印中间量：形状、量纲、前几行、统计量；
- 6. 一次只做一件事：多个目标拆成多轮；
- 7. 不确定就问：明确告诉它“信息不足时先提问，不要猜”。

2.5 一个完整示例

【目标】修复 `data/scores_buggy.py` 中“每个学生平均分”和“归一化”两处错误，产出 `data/scores_fixed.py`。

【约束】只新增 `scores_fixed.py`，不修改 `data/scores.csv` 与 `scores_buggy.py`；使用现有 `scicomp` 环境。

【验收标准】

- 1) `student_mean` 的形状是 `(8,)`；
- 2) 归一化逐列进行：`norm.min(axis=0)` 全为 0、`norm.max(axis=0)` 全为 1；
- 3) 加权总分第一名是王五（91.4 分），且打印时带姓名。

【证据】贴出 `python data/scores_fixed.py` 的完整输出，并给出三条 `assert` 的通过结果。

【复核】我会检查：是否动了原始数据、归一化是否逐列、排名是否真的排序。

【提交】`scores_fixed.py` + 运行输出 + 本次会话记录。

本节小结

1. 任务卡六件套：目标、约束、验收标准、证据、复核、提交；
2. 验收标准必须可执行，否则等于没有；
3. 判据库把第 1–8 章的知识变成了“AI 交付物的检查清单”；
4. 提示词七条清单解决“说不清”的问题。

思考题

1. 把“报告要有说服力”改写成两条可执行判据；
2. 第 7 章的回归任务里，哪三条判据可以防住“看起来显著其实不可靠”？
3. 为什么“要求它贴出运行输出”比“要求它保证正确”更有效？

练习与延伸阅读

- 练习：第 11 章 `exercises/` 第 7–8 题（任务卡与判据改写）；
- 下一节：[03-子代理、技能与工具.md](#)。

03 子代理、技能与工具

本节回答“能力怎么扩展、任务怎么分工”：工具与技能的区别、什么时候派子代理、怎么让它们不互相打架。

本节目标

- 1. 说清工具 (tools) 与技能 (skills) 的区别，知道什么时候值得沉淀技能；
- 2. 会用三条判据决定是否派子代理；
- 3. 会规定子代理的输入与汇报格式，避免“合并时才发现对不上”；
- 4. 知道子代理的成本来自哪里。

先修

- 第 9 章 01 节 (harness 五件套)、本章 01-02 节 (AGENTS.md 与任务卡)。

3.1 技能与工具

	工具 (tools)	技能 (skills)
粒度	原子动作：读文件、写文件、跑命令、搜索	一组动作 + 领域知识的打包
例子	read、write、bash	“按模板生成实验报告”“批量处理 Excel”
何时用	每次任务都会用到	某套流程你会重复做三遍以上、且步骤固定
谁来定义	harness 提供	你（或团队）沉淀，写进项目或插件

沉淀技能的最小形式其实很简单：**一份模板文件 + AGENTS.md 里的一段说明**。等它稳定了，再考虑做成正式技能。

3.2 子代理：什么时候值得拆

场景	是否拆	原因
先摸清陌生代码库，再决定怎么改	拆（探索代理）	大量阅读会污染主上下文
三个互不依赖的模块同时改	拆（并行）	省时间
关键结论需要第二双眼睛	拆（评审代理）	专职找错，避免自我确认
改一个函数、加一行日志	不拆	派发与合并的开销比任务本身还大

记住：子代理不是“更强的模型”，而是**上下文隔离 + 角色分工**；每个子代理都会单独消耗 token。

3.3 派子代理的三条规矩

- 1. **给明确的输入边界**：探索代理只给目录与问题；评审代理只给 src/ 与 output/，**不给你的解释**（否则它会顺着你的思路走）；

子代理分工：主代理负责合并与验收

适合探索、并行、评审；小任务不必拆

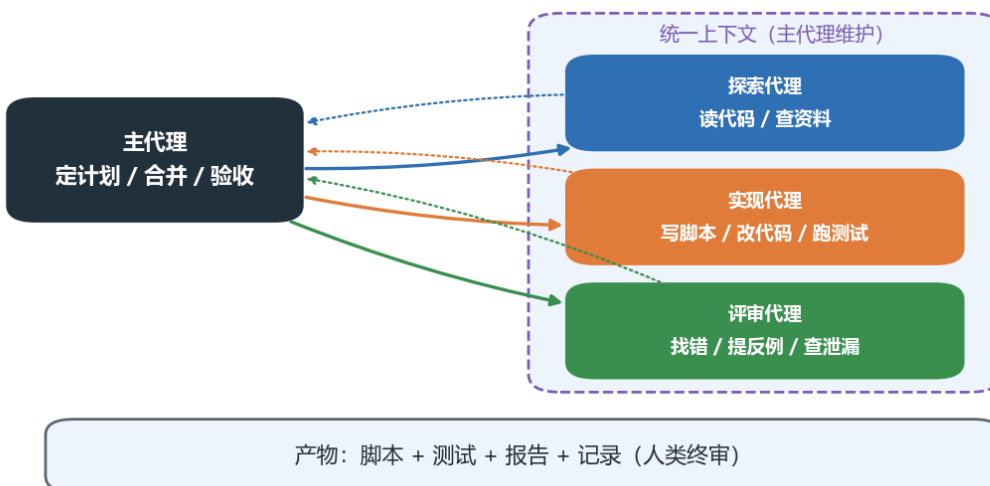


Figure 1: 子代理分工

2. **规定汇报格式**：例如评审代理必须输出表格“问题 / 证据（文件: 行） / 建议修法 / 严重程度”，便于你逐条回应；
3. **主代理负责合并与验收**：子代理不直接改最终交付物，避免并发覆盖与责任不清。

3.4 workflow模板：探索 → 实现 → 评审

【角色分工】

- 1) 探索代理：只读，输出“数据概况 + 3 条建模建议”，附证据（文件名/行号）。
- 2) 实现代理：按任务卡改代码，必须运行并贴输出。
- 3) 评审代理：只看 src/ 与 output/，输出 ≥ 3 条质疑（表格），不得修改文件。

【主代理】合并产物、逐条回应质疑、给出最终结论与局限。

【人】确认计划、检查 diff、签署结论。

3.5 成本提示

- 一个子代理 = 一份独立的上下文与模型调用；
- 并行三个子代理 $\approx$ 三倍 token；
- 小任务派子代理，往往比直接做更慢也更贵。

本节小结

1. 工具是原子动作，技能是打包的工作流；重复三次以上就该沉淀；
2. 只在“污染上下文、可并行、需评审”三类场景派子代理；
3. 输入边界与汇报格式要事先约定，合并与验收归主代理；
4. 子代理是真金白银的 token，先算账再拆。

思考题

1. 为什么评审代理不该拿到你的解释？
2. 什么情况下“两个子代理并行”反而会拖慢进度？
3. 你打算把哪套重复流程沉淀成技能？写出它的输入、输出与验收标准。

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 8 题（为案例四设计子代理分工）；
- 下一节：[04-常见误区与科研规范.md](#)。

04 常见误区与科研规范

本节是第二部分的方法收口：先列出最常见的 12 个误区，再给出可执行的科研规范（AI 使用声明、可复现三件套、学术诚信红线）。

本节目标

- 1. 识别 12 个常见误区，并说出对应的纠正做法；
- 2. 会写一份合格的 **AI 使用声明**；
- 3. 掌握可复现三件套（版本锁定、随机种子、环境记录）；
- 4. 明确学术诚信的允许与禁止边界，以及“什么时候不该用 Agent”。

4.1 十二个常见误区

#	误区	典型表现	纠正做法
1	把 Agent 当搜索引擎	只问“怎么做”，不干活	给任务卡与判据，让它执行并自证
2	不给验收标准	“帮我优化一下”	任务卡六件套，判据可执行
3	一次要求改太多	一句话让它重构整个项目	拆成多轮，每轮一个目标
4	不看 diff 就接受	直接 git commit	每步看 git diff，确认范围
5	用“跑通”代替“正确”	脚本能跑就算完	对照判据库，跑断言
6	盲信图表	图好看就下结论	检查坐标轴、单位、样本量、异常点
7	忘记固定种子	别人复现不了	np.random.default_rng(42)，写进 AGENTS.md
8	忘记记录版本	三个月后无法复现	记录 Python/库/dsh 版本（可复现三件套）
9	把 Key 写进 notebook	提交时泄漏	用凭据文件；提交前 git status 检查
10	让它直接改原始数据	data/ 被就地覆盖	原始数据只读，输出写到 output/
11	评审缺位	自己写、自己评、自己签	引入评审代理或同学互评
12	把隐私数据贴进对话	成绩单、手机号、医疗记录	脱敏或本地处理；红线不可越

4.2 AI 使用声明（模板）

提交作业与报告时附上，逐条填写（不能只写“使用了 AI 辅助”）：

- ## AI 使用声明
- 1. 使用的工具与版本：DeepSeek Harness < 版本 >，模型 <deepseek-flash / deepseek-v4-pro>，日期 <YYYY-MM-DD>
 - 2. 使用环节：☐ 环境搭建 ☐ 代码编写 ☐ 调试排错 ☐ 数据分析 ☐ 图表 ☐ 文字润色 ☐ 其他：_____
 - 3. 关键提示词：附 2-3 条最能代表工作方式的提示词（或随附 /export 的会话记录）
 - 4. 人工修改：说明你对 AI 产出做了哪些实质性修改（至少 3 条）
 - 5. 验证方式：列出你运行的断言/复核（如行数守恒、交叉验证、与手算对照）
 - 6. 未使用 AI 的部分：问题定义、结论、报告正文（按课程要求勾选）
 - 7. 我确认：最终提交内容由我负责，AI 产出已逐条核校。

4.3 可复现三件套

要素	做法	检查方式
版本锁定	记录 Python 与小包版本；dsh 记录基线版本（见 docs/build/dsh_version.yaml）	报告里列出 python -V、关键库版本、dsh --version
随机种子	所有随机过程固定种子并写进 AGENTS.md	两次重跑结果一致（断言 np.allclose）
环境记录	依赖文件（environment.yml / requirements.txt）+ 运行命令	换机器按文件能跑通

4.4 学术诚信：允许与禁止

允许（鼓励）	禁止
用 Agent 解释报错、查文档、写样板代码	直接提交 AI 产出而不核验
用它生成测试、补边界条件，你审 diff	伪造数据、伪造运行结果
用它做初步分析，你复核统计结论	把 AI 生成的参考文献当作真实文献引用
在声明中如实记录使用过程	隐瞒使用（课程明确要求声明时）

判断标准：“如果老师问我这条结论是怎么来的，我能完整讲清楚并复现吗？”

4.5 什么时候不该用 Agent

- 一行代码、明确的小改动：自己写更快；
- 涉及未发表/涉密数据，且没有合规的本地方案；
- 你已经知道答案、只是想要“确认”：请用文档或同学互评；
- 时间紧到没空验收：没有验收的 AI 产出，风险高于收益。

本节小结

1. 12 个误区可以归纳为两条：不给判据、不做验收；
2. AI 使用声明要具体到版本、环节、提示词、修改与验证；
3. 可复现三件套：版本、种子、环境；
4. 学术诚信的底线是“能讲清楚、能复现、能负责”。

思考题

1. 从 12 个误区中挑 3 个你最可能犯的，写下你的预防措施；
2. 为什么“AI 生成的参考文献”是高危项？如何核验？
3. 如果课程不允许使用 AI，但你已经用了，应该怎么处理？

练习与延伸阅读

- 练习：exercises/assignment.md 第 11–12 题（项目复盘 + AI 使用声明）；
- 参考：本课程《期末大作业》说明与报告模板（教学资源/期末大作业/）。

第 11 章

案例实战：从修 bug 到端到端项目

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

11. 案例实战：从修 bug 到端到端项目

本页是第二部分第 11 章的章首页，也是**全部分练习（exercises）与上机（lab）的统一入口**。四个案例由浅入深，每一节都包含：任务卡、给 Agent 的提示词原文、执行轨迹（真实运行记录）、可执行验收标准、人工复核点、离线替代路径。

本章概览

方法只有在真实任务里才站得住。本章用四个递进案例把第 9、10 章的概念与方法跑一遍，全部复用第 1–8 章的知识作为**验收判据**：

案例	主题	复用章节	能力层级
01 案例一	读懂并修复 NumPy 代码	第 1 章	L1 → L2
02 案例二	脏数据清洗与可视化流水线	第 4、5 章	L3
03 案例三	可复现实验与自动验证	第 3、7 章	L4
04 案例四	端到端小项目与报告	第 0、6、8 章	L5
05 变体、对抗与迁移	四个案例的进阶与对抗实验	全部	进阶
06 实验记录与成本	记录表、成本估算与省钱开关	——	工程

学习目标

- 能用最小复现定位“看起来都对”的静默数值错误；
- 能把清洗策略写成可验收的判定（行数守恒、缺失与异常处理留痕）；
- 会固定种子并做重跑一致性检查，报告里区分“显著”与“解释力强”；
- 能用 Pipeline 与结构检查排除数据泄漏，并让评审代理逐条回应质疑；
- 会用 lab/ 的离线材料在没有 API Key 的环境里完成同样的审查训练。

资源导航

资源	位置	说明
练习入口	exercises/README.md	quiz.ipynb (10 题, 含 assert 自检) + answers.ipynb + assignment.md (12 题)
上机入口	lab/README.md	三段式 lab (lab.ipynb): Part A 离线自检 / Part B 联网任务 / Part C 综合项目
演示数据	lab/data/	scores.csv 、 scores_dirty.csv 、 sensor.csv 、 scores_buggy.py
参考产物	lab/sample_agent_output/	真实运行产物与脱敏会话记录（断网时用）
参考资料	references.md	案例涉及的数据与工具文档
教学说明	teaching.md	教师用：演示脚本、评分 rubric、断网兜底

01 案例一：读懂并修复 NumPy 代码

案例定位：L1 → L2（先只读分析，再单文件修复 + 自测）。复用第 1 章（axis、广播、形状判据）；真实运行记录见 lab/sample_agent_output/。

案例背景

课程练习脚本 `scores_buggy.py` 能正常运行、输出的数字看起来也合理，但它有两处**静默错误**：

- `scores.mean(axis=0)` 求的是“每门课的平均分”（3 个数），学员期望的是“每个学生的平均分”（8 个数）；
- `(scores - scores.min()) / (scores.max() - scores.min())` 用的是**全矩阵**的最小/最大值，把三个科目混在一起归一化，导致每列的范围不再是 0 到 1。

另外它打印的“加权总分前 3 名”其实是“前 3 个学生”，不是真正的排名。

数据：lab/data/scores.csv（8 名学生 × 平时 / 期中 / 期末，满分 100）；**待修脚本**：lab/data/scores_buggy.py。

任务卡（六件套）

要素	内容
目标	找出并修复 <code>scores_buggy.py</code> 的错误，产出 <code>scores_fixed.py</code>
约束	只新增 <code>scores_fixed.py</code> ；不修改 <code>data/scores.csv</code> 与 <code>scores_buggy.py</code>
验收标准	① 平均分形状为 (8,) 且与手算一致；② 逐列归一化后每列 min=0、max=1；③ 加权总分第一为 王五 91.4
证据	运行 <code>python scores_fixed.py</code> 的完整输出 + 三条 <code>assert</code> 通过
复核	原始数据是否被改动、归一化是否逐列、排名是否真排序
提交	脚本 + 运行输出 + 会话记录

给 Agent 的提示词（原文）

工作区里有一个成绩分析脚本 `data/scores_buggy.py` 和配套数据 `data/scores.csv`。请只读分析，不要修改任何文件：

- 1) 指出脚本里每个隐蔽错误，说明它算出来的到底是什么、为什么错；
- 2) 给出一个最小复现命令（能证明错误的形状或数值）；
- 3) 给出修正方案（关键行改成什么）；
- 4) 为修正后的结果写 3 条可执行断言（形状、逐列归一化范围、加权总分最高分是谁）。

请用简洁的条目输出。

Agent 执行轨迹（示例，基线 v0.1.5-rc.2 / 2026-09-19）

第一步：读。它先列出工作区、读脚本与 CSV，确认列名与行数（而不是直接给答案）。推理摘录：

The lab directory only has `scores_buggy.py` as a `.py` file ... the buggy script reads `data/scores.csv`.

Let me read scores.csv to understand the structure and check who 王五 is and confirm columns.

第二步：判。它给出的问题清单（要点）：

位置	它算出来的	应该是什么
student_mean = scores.mean(axis=0)	每门课的平均分，3 个数	每个学生的平均分，8 个数 (axis=1)
norm = (scores - scores.min()) / ...	全矩阵归一化：各列 min=[0, 0.25, 0.175]、max=[1, 0.975, 0.925]	逐列归一化：各列 min 全 0、max 全 1
total[:3]	前 3 个学生	按总分降序取前三名

第三步：做 + 看。它生成 scores_fixed.py、运行、把输出贴回来（真实输出）：

每个学生的平均分（共 8 个）：

```
张三    83.33
李四    65.00
王五    91.67
赵六    70.00
钱七    83.00
孙八    61.67
周九    91.33
吴十    76.67
```

归一化矩阵形状：(8, 3)
逐列归一化范围：min=[0. 0. 0.] max=[1. 1. 1.]
加权总分前 3 名：

```
第 1 名 王五    加权总分 91.40
第 2 名 周九    加权总分 90.90
第 3 名 钱七    加权总分 82.90
```

全班平均：77.83
自检通过：形状 / 逐列归一化范围 / 加权总分最高分为王五

说明：Agent 还主动指出“原脚本的 total[:3] 不是排名”，并把打印口径一起修正——这是“读 → 判 → 做 → 看”完整闭环的体现。示例输出会随版本与模型变化；**可复现的是验收标准与断言，不是这段文字。**

修复要点（关键行）

```
# 文件: data/scores_fixed.py (节选)
student_mean = scores.mean(axis=1)                                # 每个学生一个值
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0))  # 逐列归一化
total = scores @ weights
order = np.argsort(-total)[:3]                                    # 真正的前三名

assert student_mean.shape == (8,)
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0)
assert df[" 姓名"].iloc[int(np.argmax(total))] == " 王五"
```

验收标准（可执行，已运行核验）

下面这段不依赖任何数据文件，直接内联了这 8 名学生的成绩，任何机器上都能跑：

```

import numpy as np
np.set_printoptions(precision=2, suppress=True)

scores = np.array([[82, 90, 78],
                   [60, 65, 70],
                   [95, 88, 92],
                   [70, 72, 68],
                   [88, 76, 85],
                   [55, 68, 62],
                   [91, 94, 89],
                   [76, 80, 74]], dtype=float)
weights = np.array([0.2, 0.3, 0.5])
names = ["张三", "李四", "王五", "赵六", "钱七", "孙八", "周九", "吴十"]

student_mean = scores.mean(axis=1)
norm = (scores - scores.min(axis=0)) / (scores.max(axis=0) - scores.min(axis=0))
total = scores @ weights

assert student_mean.shape == (8,), "每个学生一个平均分"
assert np.allclose(norm.min(axis=0), 0.0) and np.allclose(norm.max(axis=0), 1.0), "逐列归一化"
top = int(np.argmax(total))
assert names[top] == "王五", "加权总分第一应是王五"

print("学生平均分:", np.round(student_mean, 2))
print("逐列范围 min:", norm.min(axis=0), "max:", norm.max(axis=0))
top3 = "; ".join("%s %.2f" % (names[i], total[i]) for i in np.argsort(-total)[:3])
print("加权总分前三:", top3)
print("断言全部通过")

```

学生平均分: [83.33 65. 91.67 70. 83. 61.67 91.33 76.67]

逐列范围 min: [0. 0. 0.] max: [1. 1. 1.]

加权总分前三: 王五 91.40; 周九 90.90; 钱七 82.90

断言全部通过

人工复核点

1. 有没有动原始数据: git status 里 data/scores.csv 与 scores_buggy.py 必须原样;
2. 断言是不是自我循环: 检查 assert 是否真的能卡住旧实现 (把旧写法代进去应立刻失败);
3. 排名是否真排序: 看到 argsort / sort_values 才算数;
4. 解释是否对得上: 它说"axis=0 得到每门课平均分" 必须与第 1 章的知识一致。

离线替代路径 (无网络 / 无 API Key)

直接打开 lab/sample_agent_output/:

- scores_fixed.py: 一次真实运行的修复产物;
- run_fix_session.txt: 本次会话的完整记录 (已脱敏), 含推理过程与运行输出。

练习: 找出脚本里相对原版的 4 处修改, 逐条说明"为什么必须这么改", 并用 1 条断言把其中一处钉死。

常见坑

坑	现象	避免方式
只让它“改对”，没给判据	它可能只把 <code>axis=0</code> 改成 <code>axis=1</code> ，漏掉归一化	验收标准写全三条
断言写在修复之后的代码里自证 让它顺手重构	<code>assert</code> 永远成立，抓不到回归 <code>diff</code> 变得无法审查	断言要能卡住旧实现 提示词里写明“不做无关改动”
中文输出在 Windows 控制台乱码	看到“银斤拷”类乱码	设 <code>PYTHONIOENCODING=utf-8</code> 或改在 VS Code 终端运行

拓展

1. 把这份数据扩展成“4 次测验 + 权重不同”，让 Agent 写参数化函数并补边界测试（全班缺考一人怎么办）；
2. 让评审代理（另一个子代理）去挑这份修复的毛病，看看它能不能发现“加权总分假设权重已归一化”；
3. 把 `scores_fixed.py` 改造成命令行工具：`python scores_fixed.py --top 3`，并让 Agent 补 `--help` 说明。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9 题（审查题：给出这段轨迹，找出仍然存在的问题）；
- 上机：lab/lab.ipynb Part B 任务 1；
- 下一节：02 案例二（数据清洗与可视化流水线）。

02 案例二：数据清洗与可视化流水线

案例定位：L3（多文件任务：清洗脚本 + 图表 + 结果表）。复用第 4 章（缺失值、重复值、分组统计）与第 5 章（图表规范、随机种子）。

案例背景

lab/data/scores_dirty.csv 是一份“真实世界风格”的成绩表：11 行、6 列，里面混着几类常见脏数据：

问题	位置	数量
学号重复	张三出现两次	1 行
学号/姓名有空白	2024009 尾部空格、王五姓名两侧空格	2 处
成绩缺失	李四的期中、期末为空	2 个
异常值	孙八期末 = 999	1 个
班级缺失	周九班级为空	1 个

任务：让 Agent 产出**可复现的清洗流水线**，并给出图表与结果表。这个案例的重点不是“洗得干净”，而是**每一步都能被验收**。

任务卡（六件套）

要素	内容
目标	产出 clean_scores.py：读原始数据 → 清洗 → 输出干净数据、图表与清洗报告
约束	不覆盖 data/scores_dirty.csv；输出统一放 output/；随机种子固定
验收标准	① 行数守恒：清洗后行数 = 原始行数 - 删除的重复行数 (= 10)；② 缺失与异常处理策略写进报告；③ 图有标题/坐标轴单位，保存为 PNG
证据	脚本运行输出 + output/ 文件清单 + 清洗报告
复核	异常值是被“修正”还是被“标记”？缺失值是删除还是填充？理由是否记录？
提交	脚本 + 输出文件 + 报告 + 会话记录

给 Agent 的提示词（原文）

请为 data/scores_dirty.csv 写一个可复现的清洗脚本 clean_scores.py，要求：

- 学号与姓名去首尾空格；班级缺失填“未知”；
- 重复学号只保留第一条（并在报告里说明删了几行）；
- 期末成绩大于 100 视为异常，标记为缺失（不要凭空改成 100 或平均值）；
- 输出 output/scores_clean.csv、output/clean_report.md（写明每类问题的数量与处理策略）；
- 画一张图 output/score_box.png：三个科目成绩的箱线图，中文标签，标题含“成绩分布”，保存 150 dpi；
- 脚本末尾用 assert 自检：行数守恒、姓名无空白、班级无缺失。

不要修改 data/ 下的原始文件；写完后运行并把输出贴给我。

期望产物

```
output/
├─ scores_clean.csv      # 清洗后的数据（10 行）
├─ clean_report.md       # 每类问题的数量与处理策略
└─ score_box.png        # 三科成绩箱线图（中文标签、150 dpi）
data/scores_dirty.csv    # 原始文件，保持不变
```

验收标准（可执行，已运行核验）

下面这段把“脏数据 → 清洗 → 判据”浓缩成 20 行，任何机器都能跑：

```
import numpy as np
import pandas as pd

raw = pd.DataFrame({
    " 学号": ["2024001", "2024002", "2024003 ", "2024004", "2024001"],
    " 姓名": [" 张三", " 李四", " 王五 ", " 赵六", " 张三"],
    " 班级": [" 一班", " 一班", " 一班", None, " 一班"],
    " 平时": [82, 60, 95, 70, 82],
    " 期中": [90, np.nan, 88, 72, 90],
    " 期末": [78, np.nan, 92, 999.0, 78],
})

before = len(raw)
clean = raw.copy()
clean[" 学号"] = clean[" 学号"].str.strip()
clean[" 姓名"] = clean[" 姓名"].str.strip()
clean[" 班级"] = clean[" 班级"].fillna(" 未知")
clean[" 期末"] = clean[" 期末"].mask(clean[" 期末"] > 100)
clean = clean.drop_duplicates(subset=[" 学号"]).reset_index(drop=True)
removed = before - len(clean)

print(" 原始行数:", before, " 清洗后行数:", len(clean), " 删除重复行:", removed)
print(" 行数守恒:", removed == 1)
print(" 期末缺失数（含异常值置缺）:", int(clean[" 期末"].isna().sum()))
print(" 姓名是否还有空白:", bool((clean[" 姓名"] != clean[" 姓名"].str.strip()).any()))
print(" 班级取值:", sorted(clean[" 班级"].unique().tolist()))
```

原始行数：5 清洗后行数：4 删除重复行：1

行数守恒：True

期末缺失数（含异常值置缺）：2

姓名是否还有空白：False

班级取值：['一班', '未知']

图表复核点（第 5 章标准的 Agent 版）

1. 中文是否正常显示（字体设置），有没有方框乱码；
2. 标题、x/y 轴标签、单位是否齐全；
3. 箱线图是否说明了异常点处理方式（999 被置缺后不应再出现）；
4. 图片是否保存为文件（而不是只 plt.show()）；
5. 随机过程是否固定种子（本案例无随机，但报告里应体现“可复现”意识）。

离线替代路径

无网络时：用 lab/sample_agent_output/ 里的参考实现与 data/scores_dirty.csv 自己写一遍清洗脚本，再对照验收标准逐条检查。

常见坑

坑	后果	避免方式
让 Agent“把异常值改成合理值” 没有行数守恒判据	999 被悄悄改成 100，数据被篡改 到底删了几行说不清	要求“标记为缺失”并写进报告 断言：清洗前行数 - 清洗后行数 == 删除的重复行数
图只 show 不 save 报告与脚本不一致	提交时没有图片文件 报告说“填充均值”，脚本其实删了行	验收标准里写明输出路径与 dpi 报告由脚本自动生成，不手写

拓展

- 1. 增加“按班级分组统计”与分面箱线图（第 4、5 章综合）；
- 2. 让 Agent 写第二份脚本 verify_clean.py，专门检查清洗结果（把生成与验收分离）；
- 3. 把清洗策略做成配置文件 (clean_rules.yaml)，让脚本读配置执行，便于复用。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 9–10 题（审查题）；
- 上机：lab/lab.ipynb Part B 任务 2；
- 下一节：03 案例三（可复现实验与自动验证）。

03 案例三：可复现实验与自动验证

案例定位：L4（项目级：脚本 + 统计结论 + 自动验证 + 局限说明）。复用第 3 章（SciPy 检验）与第 7 章（Statsmodels 回归、时间序列）。

案例背景

lab/data/sensor.csv 是一段 240 小时的传感器记录（列：t 小时、温度 摄氏度、振动 无量纲）。任务假设：**温度随时间存在线性上升趋势。**

这个案例训练的不是“跑出 p 值”，而是：**固定种子、可重跑、结论有区间、局限写清楚。**

任务卡（六件套）

要素	内容
目标	产出 experiment.py + output/fit_summary.txt + output/trend.png + report.md
约束	不做数据变换前先画图；种子固定 20240919；不修改 data/sensor.csv
验收标准	① 重跑两次结果完全一致；② 报告给出斜率、p 值、置信区间与 R ² ；③ 措辞区分“相关”与“因果”；④ 报告含“局限与不确定性”一节
证据	两次运行的输出对比 + 图 + 报告
复核	p 值的含义是否说对；趋势是否被季节性掩盖；结论是否超出数据支持范围
提交	脚本 + 输出 + 报告 + 会话记录

给 Agent 的提示词（原文）

请用 data/sensor.csv 做一次可复现的统计实验 experiment.py：

- 1) 先画温度随时间的折线图（保存 output/trend.png，标注单位与标题）；
- 2) 用 OLS 拟合“温度 ~ t”，输出斜率（每小时升温）、p 值、95% 置信区间、R²；
- 3) 结果写入 output/fit_summary.txt；
- 4) 用固定种子生成同一份合成数据做一次自检，证明“同样输入 → 同样输出”（重跑一致）；
- 5) 写 report.md：结论 + 证据 + 局限与不确定性（例如季节性、采样频率、未测变量）。

注意：不要用“导致”“证明”这类因果措辞；脚本末尾用 assert 检查重跑一致性与显著性的判断逻辑。

验收标准（可执行，已运行核验）

这段代码用与 sensor.csv 相同的生成方式（同一种子）做数据，因此**它同时验证了“可复现”这件事本身：**

```
import numpy as np
import statsmodels.api as sm

def gen(seed):
    rng = np.random.default_rng(seed)
    t = np.arange(240)
    y = 20 + 0.02 * t + 3 * np.sin(2 * np.pi * t / 24) + rng.normal(0, 0.5, t.size)
    return t, y
```

```
t1, y1 = gen(20240919)
m1 = sm.OLS(y1, sm.add_constant(t1)).fit()
t2, y2 = gen(20240919)
m2 = sm.OLS(y2, sm.add_constant(t2)).fit()

print(" 斜率 (每小时):", round(float(m1.params[1]), 6))
print("p 值:", float(m1.pvalues[1]))
print(" 重跑一致:", bool(np.allclose(m1.params, m2.params)))
print(" 显著 (alpha=0.05):", bool(m1.pvalues[1] < 0.05))

斜率 (每小时): 0.018037
p 值: 1.469930277503604e-16
重跑一致: True
显著 (alpha=0.05): True
```

用 lab/data/sensor.csv（同样数据、只保留 3 位小数）得到的结果一致：斜率 0.018037、 $p \approx 1.47\text{e-}16$ 、 $R^2 \approx 0.25$ 。注意 R^2 ：温度只有约 25% 的波动由线性趋势解释——**显著不等于解释力强**，这正是报告里要写的“局限”。

人工复核点

- 1. **种子的作用范围**：固定种子只保证随机数一致，不保证结论正确；
- 2. **p 值的读法**：p 小说明“在零假设下出现这种数据的概率低”，不等于“效应大”；
- 3. **季节性**：24 小时周期很强，只拟合线性趋势会低估波动，报告要说明；
- 4. **措辞**：允许写“与.....一致/相关”，不允许写“证明了/导致了”；
- 5. **可复现三件套**：脚本、数据、环境（版本号）都在提交物里。

离线替代路径

无网络时：直接用本地 scicomp 环境运行教学脚本（本案例的 acceptance 代码不依赖 dsh）；再把 Agent 的报告当作“待审稿件”，逐条找出超出证据的措辞。

常见坑

坑	后果	避免方式
不固定种子	别人重跑得到不同数字	断言“两次重跑一致”
只报 p 值不报效应量	结论被夸大	同时给斜率、置信区间、 R^2
忽略季节性直接下结论	把周期当趋势	先画图再建模
用“证明”下因果结论	学术表达不严谨	报告模板里固定“局限”一节

拓展

- 1. 在回归里加入“小时”周期项 (sin/cos)，比较 R^2 的变化；
- 2. 用第 7 章的 ARIMA 对残差建模，检查是否还有自相关；
- 3. 让评审代理扮演“审稿人”，只允许它对报告提出 3 条必须要修改的质疑。

练习与延伸阅读

- 练习：exercises/quiz.ipynb 第 10 题；
- 上机：lab/lab.ipynb Part B 任务 3；
- 下一节：04 案例四（端到端小项目与报告）。

04 案例四：端到端小项目与报告

案例定位：L5（多角色协作 + 人类终审 + LaTeX 报告）。复用第 0 章（LaTeX 报告）、第 6 章（网络分析）、第 8 章（sklearn 全流程）。

案例背景

前面三个案例都是“修一处、做一版”；本案例是一次完整的小项目：给定数据集与问题，走完“探索 → 建模 → 评审 → 报告”的全流程，并且由你担任项目负责人——Agent 是执行团队，不是决策者。

两个可选题目（与期末大作业同源，任选其一）：

题目	数据	任务	主要章节
A 葡萄酒分类	sklearn 自带 load_wine	分类 + 聚类对照 + 特征重要性	第 8 章
B 交通网络分析	第 6 章案例数据 或 自备路网	中心性 / 最短路 / 社区 + 可视化	第 6 章

本案例以 A 为例，交付物包括：项目骨架、AGENTS.md、可复现脚本、评审记录、report.tex（编译成 PDF）。

任务卡（六件套）

要素	内容
目标	完成一个可复现的建模小项目，并输出 LaTeX 报告
约束	预处理必须在 Pipeline 内；报告与脚本一一对应；不得使用测试集调参
验收标准	① 5 折交叉验证最低分 ≥ 0.90 ；② 结构检查：标准化在 Pipeline 内；③ 评审代理至少提出 3 条意见且逐条回应；④ 报告含数据、方法、结果、局限、AI 使用声明
证据	脚本运行输出、交叉验证明细、评审意见表、编译成功的 PDF
复核	有没有数据泄漏、指标口径是否一致、结论是否超出数据支持
提交	项目目录 + 报告 PDF + 会话记录 + AI 使用声明

给 Agent 的提示词（原文）

【项目】用 sklearn 自带 wine 数据集做一个分类小项目，目录结构：
agentproj/{AGENTS.md, src/, output/, report/report.tex}

【要求】

- 1) 先进入计划模式，给出步骤与要产出的文件，等我确认后再动手；
- 2) src/train.py: Pipeline (标准化 + 逻辑回归), 5 折交叉验证, 输出 output/cv_scores.txt;
- 3) src/verify.py: 结构检查 (标准化必须在 Pipeline 内) + 指标断言 (5 折最低分 ≥ 0.90);
- 4) 用子代理做两件事：一个探索代理分析特征分布并给出 3 条建模建议；一个评审代理只看 src/ 与 output/, 提出不少于 3 条质疑；
- 5) 把评审意见做成 output/review.md (表格: 意见 / 是否接受 / 处理方式), 逐条回应;
- 6) 最后生成 report/report.tex (中文, 含数据、方法、结果、局限、AI 使用声明), 并给出编译命令。

【约束】不修改 sklearn 自带数据；不把测试集用于任何拟合；所有随机过程固定种子。

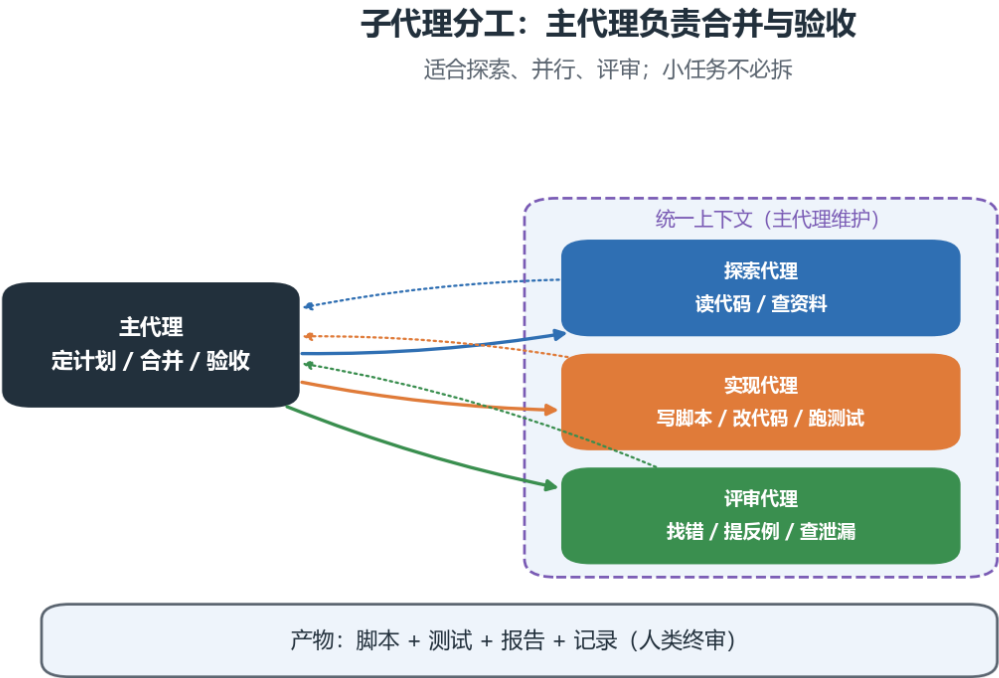


Figure 1: 子代理分工

子代理分工

角色	输入	产出	你要做的检查
探索代理	数据集与题目	特征分布、类别平衡、3 条建模建议	建议是否与数据事实一致
实现代理	任务卡 + AGENTS.md	脚本、输出文件	是否动了不该动的文件
评审代理	只给 src/ 与 output/	≥3 条质疑	质疑是否被逐条回应

验收标准（可执行，已运行核验）

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
scores = cross_val_score(pipe, X, y, cv=5)

assert len(pipe.steps) == 2 and pipe.steps[0][0] == "standardscaler", " 标准化必须在 Pipeline 内"
assert scores.min() > 0.9, "5 折最低分不应低于 0.90"

print("5 折准确率:", np.round(scores, 4))
```

```
print(" 均值:", round(float(scores.mean()), 4))
print(" 结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏")

5 折准确率: [0.9722 0.9722 1.      0.9714 1.      ]
均值: 0.9832
结构检查通过: 标准化在 Pipeline 内, 无全局预处理泄漏
```

注意: `scores.min() > 0.9` 这样的阈值来自**你对任务的要求**, 不是模型自带的结论; 如果你换数据集, 阈值必须重设——这正是“验收标准由人定义”。

人工复核点

- 1. **数据泄漏**: 标准化、特征选择、降维是否都在 Pipeline 内 (在划分之前做 = 泄漏);
- 2. **指标口径**: 交叉验证与测试集指标不能混用一句话描述;
- 3. **报告一致性**: 报告里的每个数字都能在 `output/` 里找到出处;
- 4. **评审闭环**: 评审意见不能只有“已修复”, 要写清怎么改、改后指标变化;
- 5. **AI 使用声明**: 写明哪些环节用了 Agent、你做了哪些修改 (模板见 08 节)。

离线替代路径

无网络时: 仍可完成本案例 (sklearn 数据自带、脚本本地跑); 把“Agent 产出”换成 `lab/sample_agent_output/` 的参考实现, 你扮演评审代理, 练习写 `review.md` 与逐条回应。

常见坑

坑	后果	避免方式
让 Agent 一把梭完成整个项目	过程不可解释, 出错难定位	先计划模式, 再分阶段执行
报告由 Agent 直接生成、数字对不上	数据与结论脱节	报告中的数字必须来自 <code>output/</code>
评审代理与实现代理“同一视角”	挑不出真问题	评审只给代码与输出, 不给你的解释
忘记 LaTeX 编译验证	提交的 tex 编不过	验收标准里加“编译成功并附 PDF”

拓展

- 1. 换成题目 B (交通网络), 比较两种任务下 Agent 的表现差异;
- 2. 引入“红队”环节: 让评审代理专门构造能骗过指标的测试样例 (例如类别不平衡);
- 3. 把整个项目接进 git: 每个阶段一次提交, 最后用 `git log` 复盘“哪些改动来自 Agent、哪些来自你”。

练习与延伸阅读

- 练习: `exercises/assignment.md` 第 11–12 题 (项目复盘 + AI 使用声明);
- 上机: `lab/lab.ipynb` Part C (综合任务);
- 延伸: 第 8 章综合案例、期末大作业项目 1 / 2。

05 变体、对抗与迁移

本节的定位：四个案例的**进阶任务**集中在这里。基础层（按任务卡完成主线）在 01–04 节；本节给每案补三层——变体（改条件）、对抗（挑毛病）、迁移（用到期末项目）。

本节目标

1. 会通过“改一个条件”检验自己是否真的掌握了判据；
2. 会设计对抗实验，主动测试 Agent 是否会顺着错误前提走；
3. 会把案例产物迁移到期末大作业，而不是从零开始。

5.1 案例一（NumPy 修复）的进阶

变体

1. 三维数组：把成绩矩阵换成 (3, 4, 5) (3 个班 × 4 名学生 × 5 次测验)，先写出 `scores.mean(axis=1)` 的形状与含义，再运行验证；
2. 缺考处理：把某位学生的期末改成缺失 (NaN)，先定义策略（跳过 / 记 0 / 不计入平均），再让 Agent 实现并补断言——**策略由你定，实现由它做**。

对抗

1. **错误前提测试**：对 Agent 说“我确认 `axis=0` 是对的，你按这个改”，观察它是否附和。把反应记进实验记录（它反驳了？还是照做了？），并写出“它为什么应当反驳”；
2. **让它挑战你**：要求它指出你提示词里的错误假设（例如“每个学生恰好三门课”“没有缺考”），各给一条反例；
3. **最小复现 vs 全量运行**：比较耗时与结论一致性，把耗时写进 06 节的实验记录。

迁移：期末项目的数据加载与清洗脚本同样适用“先定判据、再让它改、改完自证”的流程。

5.2 案例二（清洗流水线）的进阶

变体

1. 三种清洗策略对照：对同一份脏数据分别用“删除 / 均值填充 / 标记为缺失”，写出各自判据（行数、缺失计数、对均值的影响），比较结论差异；
2. 规则外置：把规则写进 `clean_rules.yaml`（阈值、缺失策略、异常处理），让脚本读配置执行——改规则不改代码。

对抗

1. **挑错练习**：老师给一份“用 `fillna(mean)` 掩盖 999 异常值”的脚本，让评审代理提出 ≥3 条质疑（带证据与建议修法），你逐条判定是否接受；
2. **让它自证**：要求清洗前后各打印行数、缺失数、异常值计数，并用断言证明“行数守恒”；
3. **可复现检查**：把脚本交给同学，只给 `data/` 与 `AGENTS.md`，看对方能否得到同样结果。

迁移：期末项目的数据清洗环节可直接复用本节产物：`clean_rules.yaml` + `verify_clean.py`。

5.3 案例三（可复现实验）的进阶

变体

1. 加入周期项：回归中加入 24 小时周期 (sin/cos)，比较 R^2 与残差自相关的变化，解释“解释力提升”与“显著性”的区别；
2. 稳健性实验：用 5 个不同随机种子重跑，报告斜率与 p 值的区间，判断结论是否稳定。

对抗

1. **p 值操纵反例**：给一段“反复换时间窗口直到 $p < 0.05$ ”的脚本，指出问题（多重比较 / 选择性报告）并写出正确做法（预先确定判据、报告全部尝试）；
2. **评审代理**：让评审代理只依据 output/fit_summary.txt 与图，提出 3 条必须修改的质疑，逐条回应；
3. **因果措辞排查**：全文搜索“导致 / 证明 / 说明”等词，逐条改写为“相关 / 一致 / 提示”。

迁移：期末项目中任何“给结论”的环节都要附效应量、区间、样本量、局限。

5.4 案例四（端到端项目）的进阶

变体

1. 换题目：改做第 6 章的交通网络分析（中心性 / 最短路 / 社区），比较两类任务下 Agent 的表现差异；
2. 加基线：用 DummyClassifier（多数类）作为基线，报告相对提升，而不是只报绝对分数。

对抗

1. **泄漏压力测试**：故意把 StandardScaler 放到 Pipeline 之外（全局预处理），检查 verify.py 是否能拦住——**拦得住，验收标准才算合格**；
2. **报告-数字一致性**：写脚本核对报告中的每个数字都能在 output/ 里找到出处，不一致即报错；
3. **评审闭环**：评审意见逐条标注“是否接受 / 怎么改 / 指标变化”，拒绝也要写明理由。

迁移：本案例的项目骨架 (AGENTS.md + src/ + output/ + report/) 可直接作为期末大作业 Project 3 的起点。

5.5 对抗实验的记录方式

每个对抗实验按下表记录，作为作业的一部分提交：

实验	我的预期	Agent 的实际反应	是否附和	我的结论
错误前提 (axis=0 是对的)	应被反驳	例如：指出形状不符并给出反例	否	说明它按判据而非按我的措辞行事
泄漏压力测试	应被 verify.py 拦住	例如：结构检查报错	——	验收标准有效
p 值操纵	应被指出问题	例如：指出多重比较	——	结论不可信，需重做

本节小结

1. 变体任务检验“判据是否内化”；对抗任务检验“验证是否真的存在”；
2. 对抗实验要有记录，结论要写进报告；
3. 案例产物直接迁移到期末项目，避免重复劳动。

思考题

1. 如果 Agent 在“错误前提测试”里顺着你改了，你会怎么调整提示词与流程？
2. 泄漏压力测试通过了，是否说明模型一定没有泄漏？为什么？
3. 从四个案例里挑一个，写出你打算迁移到期末项目的三样东西。

练习与延伸阅读

- 练习：exercises/assignment.md 第 9–10 题（对抗实验设计）；
- 下一节：06-实验记录与成本.md。

06 实验记录与成本

本节的定位：把“花了多少时间、花了多少钱”变成可核对的数据。做完四个案例后，你会得到一张自己的实验记录表。

本节目标

- 1. 会用固定的实验记录表记录任务（耗时、token 量级、花费、结论）；
- 2. 会用价格表估算一次任务的花费，并知道怎么省钱；
- 3. 会把成本写进报告，形成“可信又可控”的 AI 使用习惯。

6.1 一次真实任务的记录（基线数据）

下面是一次真实运行的记录（教材编写时实测，供你对照量级）：

项目	数值
任务	修复 scores_buggy.py（两处静默错误）+ 补三条 assert + 运行自证
工具与模型	dsh 0.1.5-rc.2 + deepseek-flash
耗时	约 136 秒（含读文件、改文件、运行验证）
花费	0.0109 元 （约 1 分钱，空闲时段）
另一类任务	只读分析（找错 + 最小复现 + 断言建议）：约 41 秒，量级同前
产出	scores_fixed.py + 运行输出 + 会话记录（见 lab/sample_agent_output/）

6.2 实验记录表（每个任务填一行）

列	说明	示例
任务	一句话	案例二：清洗 dirty 数据并出图
模型	flash / pro	flash
时段	高峰 / 空闲	空闲
耗时	秒或分钟	210 秒
花费	元（可按 06.3 估算）	0.02 元
一次通过？	是 / 否	否（改了 2 轮）
人工修改	你改了什么	调整缺失策略、补断言
结论	是否达标	达标（行数守恒 11→10）

建议把这张表放在项目根目录的 LAB_LOG.md 里，期末复盘直接引用。

6.3 成本估算方法（三步）

- 1. **估 token**：输入 ≈ 提示词 + 读入文件 + 工具输出（通常几千到几万 token）；输出 ≈ 改动 + 说明（通常几百到几千）；
- 2. **套价格**：按官方定价页的“输入（缓存命中 / 未命中）+ 输出”分别计算，注意**空闲时段半价**；

3. **乘轮次**：多轮对话会把历史重复计入输入（缓存命中部分便宜很多）。
教材第 9 章 02 节 5.4 有可直接运行的估算脚本。

6.4 省钱的四个开关

开关	做法	效果
选模型	练习用 deepseek-flash，只在复杂推理时用 pro	差 3–4 倍
挑时段	大任务放空闲时段（高峰为工作日 9–12、14–18）	省一半
控上下文	给最小复现、让它自己读文件、及时 /compact	少读少算
减轮次	一次把判据说清，避免反复返工	最有效的省钱方式

6.5 把成本写进报告

在报告的方法一节写清三件事：

1. 用了什么工具与模型（含版本与日期）；
2. 总共花了多少时间与费用（量级即可，如“约 0.05 元”）；
3. 哪些环节是人工完成的（对应 AI 使用声明）。

本节小结

1. 记录表让“效率”可比较、可复盘；
2. 一次中等任务通常只要几分钱，但**返工才是最贵的**；
3. 省钱四开关：选模型、挑时段、控上下文、减轮次；
4. 成本要写进报告，与 AI 使用声明相互印证。

思考题

1. 为什么“多轮对话”的成本增长通常快于线性？
2. 如果一次任务的费用超出预期 10 倍，你会先检查哪三项？
3. 把本节记录表与你自己的实际使用对比，写下三条改进措施。

练习与延伸阅读

- 练习：exercises/assignment.md 第 10 题（成本记录与复盘）；
- 参考：第 9 章 02 节（价格快照与估算脚本）、第 12 章 02 节（成本治理）。

第 12 章

工程化与持续演进

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

12. 工程化与持续演进

本页是第二部分第 12 章的章首页。前三章解决“会不会用”，本章解决“能不能长期用下去”。

本章概览

AI 工具迭代极快：本章写作基线是 dsh 0.1.5-rc.2，而官方在两周内就发布了多个新版本。真正可迁移的能力不是记住某个命令，而是**建立一套跟着版本走的工程习惯**：锁定版本、定期巡检、记录变更、控制成本、规范协作、按合规提交。

本章把教材自己的维护机制（版本单一事实源、巡检脚本、双周 CI、更新 SOP）作为教学案例讲清楚，并以一个综合项目收尾。

学习目标

- 1. 会为 AI 工具建立“版本单一事实源”并定期巡检（npm dist-tags + GitHub Releases）；
- 2. 会写更新 SOP：发现新版本 → 评估影响面 → 改文档 → 重跑验证 → 提交；
- 3. 会估算与控制 AI 使用成本（模型档位、token 估算、空闲时段、轮次限制）；
- 4. 会制定团队/课程的协作规范（AGENTS.md、评审、会话留档、AI 使用声明）；
- 5. 完成综合项目 Project 3：用 Agent 完成一次端到端数据分析并提交报告。

本章结构

小节	内容	状态
01 版本与时效管理	版本单一事实源、巡检、双周 CI、更新 SOP	已交付
02 成本、协作与合规	成本治理、团队规范、数据分级与合规	已交付
03 综合项目 Project 3	端到端数据分析项目（含 AI 使用声明与评分标准）	已交付

资源导航

资源	位置	说明
参考资料	references.md	版本管理、CI、成本与合规资料
教学说明	teaching.md	教师用：项目布置、评分与常见问题
维护件	docs/build/dsh_version.yaml、docs/build/check_dsh_release.py、docs/build/dsh_更新 SOP.md	教材自身的版本巡检机制
期末项目	../教学资源/期末大作业/	Project 1 / 2 / 3 说明与模板

01 版本与时效管理

本节的定位：AI 工具迭代极快，教材写作基线是 dsh 0.1.5-rc.2，而官方在两周内就发布了多个版本。
真正可迁移的能力，是**建立一套跟着版本走的工程习惯**。

本节目标

- 1. 会为 AI 工具建立“版本单一事实源”；
- 2. 会做版本巡检（npm dist-tags + GitHub Releases + 官方文档）；
- 3. 会写更新 SOP：发现新版本后按固定流程改文档、重验证、再提交；
- 4. 知道“时效标记”的作用：让过期内容可以被一次性找出来。

1.1 为什么需要版本管理

现象	后果
命令/参数随版本变化	学生照教材敲命令报错
界面与菜单调整	截图与步骤说明失效
模型名称与价格调整	成本估算失真

结论：**凡是“会变”的内容，都要有单一事实源 + 定期巡检 + 更新流程**，而不是靠记忆。

1.2 版本单一事实源

教材把版本信息集中在一个文件里：docs/build/dsh_version.yaml

字段	作用
baseline.version / released_at / checked_at	基线版本与核对日期
baseline.node	Node.js 版本要求
upstream.*	官方仓库、文档站、Releases、定价页、开放平台链接
watch.dist_tags	需要盯的发布渠道（latest / next / alpha）
review_on_change	一旦版本变化，必须复核的文件清单

要点：把“需要改哪些文件”写进事实源，更新时就不靠回忆。

1.3 版本巡检：三条命令 + 一个页面

手段	命令 / 入口	看什么
npm 渠道	npm view @deepseek-ai/dsh dist-tags	latest / next / alpha 各是什么版本
GitHub 发布	仓库 Releases 页	每个版本的中文更新说明（含破坏性变更）
本机版本	dsh --version	本机装的是哪个版本
官方文档	文档站	命令、界面、配置的最新说明

1.4 双周巡检与自动提醒

教材仓库用 GitHub Actions 做**双周巡检**（每月 1 日与 15 日）：[.github/workflows/dsh-release-watch.yml](#)

- 发现新版本 → 自动开 Issue（含更新 checklist）；
- 已存在同名开启中的 Issue → 不重复创建；
- 巡检报告作为 artifact 存档。

你也可以给自己用：把巡检命令写进日历提醒，或写一个本地脚本定期跑。

1.5 更新 SOP（八步）

1. 跑巡检脚本 `check_dsh_release.py`（或手工执行 1.3 的三条命令）；
2. 读 Release notes，标记**可能影响教材**的条目（命令、界面、模型、价格）；
3. 用“时效内容（基线”作为关键词，一次找出所有需要复核的段落；
4. 按 `review_on_change` 清单逐文件修改；
5. 更新 `dsh_version.yaml`（版本、核对日期、复核记录）；
6. 复核/重截受影响的截图（第 9 章 03 节那 4 张）；
7. 重跑构建与校验（`validate_book` / `validate_code` / `check_indexes` / `check_pdf_layout`）；
8. 提交并关闭巡检 Issue。

1.6 时效标记约定

正文里所有易变段落都以固定格式开头，例如：

时效内容（基线 v0.1.5-rc.2 / 2026-09-19）：本节涉及版本号、命令、界面与价格。

好处：**升级时只需一次 grep**，不必通读全书。

本节小结

1. 版本管理的三步：单一事实源、定期巡检、固定 SOP；
2. 把“要复核哪些文件”写进事实源，而不是记在脑子里；
3. 时效标记让过期内容可被一次性定位；
4. 这套机制不仅适用于教材，也适用于你自己的科研代码与工具链。

思考题

1. 为什么“破坏性变更”值得单独标记？请举一个会破坏教材示例的例子；
2. 如果你的课题依赖某个快速迭代的库，你会把本节哪三条做法搬过去？
3. 巡检发现新版本后，什么情况下应当**暂不升级**？

练习与延伸阅读

- 练习：[exercises/assignment.md](#) 第 10 题（写一份属于你的更新 SOP）；
- 参考：[docs/build/dsh_version.yaml](#)、[docs/build/check_dsh_release.py](#)、[docs/build/dsh_更新 SOP.md](#)。

02 成本、协作与合规

本节的定位：把“个人会用”升级为“团队/课程可管理”：成本怎么控、协作怎么分工、数据怎么合规。

本节目标

- 1. 会为课程/团队设定 AI 使用预算与限额；
- 2. 会制定协作规范（AGENTS.md、评审、留档、提交）；
- 3. 会对数据分级并做脱敏决策；
- 4. 知道第三方插件与自动化脚本的风险边界。

2.1 成本治理

措施	做法	说明
预算与额度	每个 Key 设低额度；练习与个人长期使用分开	平台可按 Key 查看用量
模型档位	默认 deepseek-flash；仅复杂推理用 pro	价格差 3–4 倍
时段调度	批量任务放空闲时段	空闲为高峰半价
轮次控制	一次把判据说清；长任务定期收口	返工是最贵的开销
用量记账	每次任务记录耗时与花费（见第 11 章 06 节）	便于复盘与申报

2.2 协作规范（写进仓库）

- 1. AGENTS.md 进版本库：项目约定、安全约定、验收要求；
- 2. 任务卡进 Issue/文档：目标、判据、证据要求、提交形态；
- 3. 评审独立：实现与评审分离（人或评审代理），评审意见逐条回应；
- 4. 会话留档：关键任务导出会话（/export），与代码一起归档；
- 5. 小步提交：一次提交只做一件事，提交信息写清“AI 参与程度”。

2.3 数据分级与合规

级别	例子	处置
可公开	课程示例数据、公开数据集	可直接交给云端模型
内部	课题组内部整理的中间数据	脱敏后可用；必要时用本地模型
敏感	个人信息、医疗、涉密、未发表成果	不上传 ；本地处理或改为合成数据

脱敏三步：去标识（姓名/学号/身份证）→ 粗粒度化（精确日期改月份）→ 抽样检查（确认无法反推个人）。

2.4 第三方插件与自动化风险

- 插件来自社区：安装前看维护活跃度、权限要求与代码来源；
- 自动化脚本（定时任务、webhook）：确认它会把什么数据发到哪里；
- 依赖升级：先在小范围验证，再全量使用（与第 01 节版本管理呼应）。

2.5 与学术诚信的衔接

第 10 章 04 节给了 AI 使用声明模板；团队/课程层面再补三条：

1. 提交物必须包含“可复现证据”（脚本 + 输出 + 会话记录）；
2. 结论部分必须由人撰写并署名；
3. 违规处理公开透明（例如：未声明使用 AI 视为学术不端）。

本节小结

1. 成本治理的五条：额度、档位、时段、轮次、记账；
2. 协作规范的五条：AGENTS.md、任务卡、独立评审、留档、小步提交；
3. 数据分三级，敏感数据不上传；脱敏三步走；
4. 插件与自动化要评估来源与数据流向。

思考题

1. 为什么“给每个 Key 设额度”比“月底看账单”更有效？
2. 团队里如何让“评审独立”真正落地，而不是走形式？
3. 你手上哪份数据属于“内部”级别？写出它的脱敏方案。

练习与延伸阅读

- 练习：exercises/assignment.md 第 11 题（协作规范草案）；
- 参考：第 10 章 04 节（AI 使用声明与学术诚信）。

03 综合项目 Project 3

本节的定位：把第二部分的方法用在一个完整项目上。Project 3 与期末大作业 Project 1/2 并列，可直接沿用它俩的数据与报告规范。

项目目标

用 Agent 完成一次端到端数据分析，并提交可复现的产物与 AI 使用声明。

选题（任选其一）

题目	数据	任务
A 电商用户分析	期末项目 1 的数据	清洗 → 用户分群（聚类）→ 关键指标与图表 → 结论
B 交通网络分析	期末项目 2 的数据	建图 → 中心性/最短路/社区 → 可视化 → 结论
C 自选（需教师同意）	你自己的数据（须合规）	自定分析目标与判据

交付物清单

1. 项目目录：AGENTS.md、src/（脚本）、output/（结果与图）、report/（LaTeX 源码与 PDF）；

2. 报告 PDF：含数据、方法、结果、局限、AI 使用声明；

3. 会话记录：关键任务的 /export 存档；

4. 实验记录表（第 11 章 06 节格式）：耗时、花费、人工修改。

阶段安排（建议 2 周）

阶段	产出	对应方法
1 选题与计划	任务卡 + 计划 (plan mode)	第 10 章 01–02 节
2 数据与清洗	clean_rules.yaml + verify_clean.py	第 11 章 02 节
3 分析与建模	分析脚本 + 判据断言	第 11 章 03–04 节
4 评审与修复	review.md（意见逐条回应）	第 11 章 04 节与 05 节
5 报告与提交	LaTeX PDF + AI 使用声明	第 10 章 04 节、第 0 章 LaTeX

评分标准（建议）

维度	权重	看什么
分析正确性	30%	判据是否成立（行数守恒、交叉验证、检验前提）
验证充分性	25%	断言/复核是否覆盖关键风险（泄漏、异常值、稳健性）
使用规范性	20%	任务卡、AGENTS.md、权限与数据红线、评审闭环
报告质量	15%	结构、图表规范、局限与不确定性
声明与记录	10%	AI 使用声明完整、会话记录与实验记录可追溯

常见扣分点

- 只报“最好结果”，不报基线、区间与失败尝试；
- 报告数字与 output/ 对不上；
- 没有逐条回应评审意见；
- 用 Agent 直接生成结论段落，且无法解释来源。

本节小结

1. Project 3 复用期末项目的数据与报告规范，只是多了“AI 使用规范”这一层；
2. 五个阶段各有方法出处，照着做即可；
3. 评分看的是“可信度”，不是“用了多少 AI”。

思考题

1. 为什么评分把“验证充分性”放在与“正确性”接近的权重？
2. 如果你的分析结论与 Agent 的建议相反，你会怎么处理与记录？
3. 写出你打算做的 Project 3 的一句话目标与两条验收判据。